



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Программное обеспечение ЭВМ и информационные технологии

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Оценка кредитоспособности физических лиц в банковских
системах с использованием алгоритма градиентного бустинга деревьев
решений

Студент ИУ7-85Б

(подпись)

Е. Фролов

Руководитель ВКР

(подпись)

Н.В. Новик

Нормоконтролер

(подпись)

(инициалы, фамилия)

2026 год

РЕФЕРАТ

Расчетно–пояснительная записка 86 с., 19 рис., 6 табл., 27 ист., 3 прил.

Работа посвящена разработке метода оценки кредитоспособности физических лиц с применением алгоритма градиентного бустинга деревьев решений.

Ключевые слова: кредитный скоринг, кредитный риск, машинное обучение, градиентный бустинг, GBDT, классификация, банковские системы.

Рассмотрена задача оценки кредитоспособности заемщиков в банковских системах. Проведен анализ предметной области кредитного скоринга, а также рассмотрены существующие методы, включая традиционные статистические подходы и современные алгоритмы машинного обучения. Выполнен обзор существующих скоринговых систем и проведен сравнительный анализ методов.

Сформулирована цель работы и выполнена формализация постановки задачи с использованием IDEF0-диаграммы.

Метод оценки кредитоспособности разработан на основе алгоритма градиентного бустинга деревьев решений. Описаны его основные этапы: подготовка данных, обучение модели и интерпретация результатов. Определены структуры данных и схема взаимодействия компонентов системы.

Обоснован выбор инструментов программной реализации. Разработано программное обеспечение, реализующее предложенный метод, описаны входные и выходные данные, а также интерфейсы взаимодействия системы.

Проведено тестирование разработанного программного обеспечения и исследование его эффективности. Выполнено сравнение результатов с традиционными методами кредитного скоринга и проанализировано влияние различных факторов на точность предсказаний.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	8
ВВЕДЕНИЕ	9
1 Аналитический раздел	11
1.1 Анализ предметной области	11
1.2 Градиентный бустинг деревьев решений	14
1.3 Логистическая регрессия	17
1.4 Линейный дискриминантный анализ	20
1.5 Метод опорных векторов	22
1.6 Классификация существующих решений	26
1.7 Скоринговая система FICO	28
1.8 Скоринговые решения Experian	29
1.9 Скоринговые системы на основе машинного обучения	30
1.10 Формулировка цели	31
1.11 Формализация задачи. IDEF0-диаграмма	32
2 Конструкторский раздел	35
2.1 Требования к разрабатываемому методу	35
2.2 Требования к программному комплексу	36
2.3 Основной алгоритм работы	36
2.4 Описание входных данных и признакового пространства	38
2.5 Предобработка данных	40
2.6 Параметры и настройка модели	41
2.7 Формирование результата и принятие решения	42
2.8 Структуры данных	43
2.9 Взаимодействие компонентов системы	44
3 Технологический раздел	47
3.1 Обоснование выбора технологий	47
3.2 Основные программные модули системы	48
3.3 Формат входных и выходных данных	51

3.4	Тестирование программного обеспечения	51
4	Исследовательский раздел	61
4.1	Анализ дисбаланса классов	62
4.2	Оптимизация гиперпараметров модели	63
4.3	Сравнение моделей по основным метрикам	66
4.4	Сравнение моделей по дополнительным метрикам	67
4.5	Анализ ROC-кривых	69
4.6	Анализ Precision-Recall кривых	70
4.7	Анализ значимости признаков	71
4.8	Ограничения проведенного исследования	72
	ЗАКЛЮЧЕНИЕ	75
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	77
	ПРИЛОЖЕНИЕ А	80
	ПРИЛОЖЕНИЕ Б	83
	ПРИЛОЖЕНИЕ В	86

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

1. Кредитоспособность – способность заемщика своевременно и в полном объеме выполнять свои кредитные обязательства.
2. Кредитный риск – вероятность невыполнения заемщиком обязательств по кредиту.
3. Дефолт (от англ. Default) – ситуация, при которой заемщик не выполняет свои обязательства по возврату кредита.
4. Машинное обучение (от англ. Machine Learning, ML) – раздел искусственного интеллекта, изучающий методы построения моделей на основе данных.
5. Градиентный бустинг деревьев (от англ. Gradient Boosting Decision Trees, GBDT) – ансамблевый метод машинного обучения, основанный на последовательном построении деревьев решений.
6. Логистическая регрессия (от англ. Logistic Regression) – статистический метод, используемый для решения задач бинарной классификации.
7. Метод опорных векторов (от англ. Support Vector Machine, SVM) – алгоритм машинного обучения для задач классификации и регрессии.
8. Признак – характеристика объекта, используемая в модели машинного обучения.
9. Целевая переменная (от англ. Target Variable) – переменная, значение которой требуется предсказать.
10. Обучающая выборка (от англ. Training Dataset) – набор данных, используемый для обучения модели.
11. Переобучение (от англ. Overfitting) – ситуация, при которой модель хорошо работает на обучающих данных, но плохо обобщает на новых данных.
12. Скоринговый балл – числовая оценка уровня кредитного риска заемщика.

ВВЕДЕНИЕ

Актуальность темы исследования обусловлена современными тенденциями в банковском секторе, где эффективность принятия кредитных решений зависит от точности и скорости скоринговых методов. Скоринг является ключевым инструментом для разделения потенциальных заемщиков на «хороших» – тех, кто с высокой вероятностью вернет кредит, и «плохих» – тех, кто, скорее всего, не вернет кредит. Эти термины общеприняты в данной области и отражают степень кредитоспособности клиентов [1].

В скоринге используются косвенные признаки, поскольку прямые признаки надежности заемщика отсутствуют. Оцениваются сведения о заемщике, включая уровень дохода, наличие действующих кредитных обязательств, возраст, трудовой стаж и другие значимые характеристики. На их основе строится комплексный критерий для принятия решения о выдаче кредита.

В последние годы динамика кредитования физических лиц в России характеризуется существенными изменениями. По данным Frank RG, в 2025 году объем выданных кредитов физическим лицам составил около 9,89 трлн рублей, что на 25,6% ниже уровня 2024 года. Это стало минимальным значением за последние шесть лет. [2] Одновременно наблюдается снижение спроса на кредитные продукты на фоне ужесточения денежно-кредитной политики. При этом сохраняется значительный уровень просроченной задолженности, что повышает кредитные риски банковской системы и усиливает требования к точности скоринговых моделей.

Распространение современных финансовых технологий привело к трансформации рынка финансовых услуг и формированию новых моделей финансирования, таких как краудфандинг, краудинвестинг, P2P-кредитование и другие. Для сохранения конкурентоспособности банки вынуждены совершенствоваться, повышая скорость обработки заявок и оценки кредитоспособности заемщиков.

Актуальность работы состоит в необходимости повышения точности и эффективности оценки кредитоспособности заемщиков в условиях роста объемов кредитования и увеличения рисков. Современные банковские системы требуют автоматизированных решений, способных обрабатывать большие объемы данных и учитывать различные факторы, влияющие на вероятность дефолта.

Традиционные методы оценки кредитного риска не всегда обеспечивают достаточную точность, что приводит к финансовым потерям. В связи с этим возрастает интерес к методам машинного обучения. Их применение позволяет повысить качество прогнозирования и адаптироваться к изменяющимся условиям.

Цель работы заключается в разработке модели оценки кредитоспособности заемщика на основе методов машинного обучения, обеспечивающей высокую точность прогнозирования кредитного риска.

Для достижения поставленной цели требуется решить следующие задачи:

- изучить существующие методы оценки кредитоспособности заемщиков;
- провести анализ методов машинного обучения, применяемых в скоринге;
- разработать модель оценки кредитного риска на основе градиентного бустинга;
- реализовать программную модель и провести ее обучение;
- оценить качество разработанной модели и сравнить с альтернативными методами.

1 Аналитический раздел

В данном разделе проводится анализ предметной области оценки кредитоспособности физических лиц в банковских системах. Описываются основные понятия, используемые в кредитном скоринге, а также рассматриваются современные методы машинного обучения, применяемые для оценки кредитного риска, включая алгоритм градиентного бустинга деревьев решений. Выполняется обзор существующих решений в области кредитного скоринга и приводится сравнительный анализ традиционных статистических методов и алгоритмов машинного обучения. На основании проведенного анализа формулируется цель работы и осуществляется формализация постановки задачи в виде IDEF0-диаграммы.

1.1 Анализ предметной области

Кредитный риск представляет собой вероятность того, что заемщик или контрагент банка не выполнит свои обязательства в соответствии с согласованными условиями. Для банков кредитный риск является одним из ключевых видов риска, поскольку его реализация может привести к финансовым потерям и ухудшению качества кредитного портфеля. Управление кредитным риском предполагает не только оценку отдельных заемщиков, но и контроль совокупного уровня риска по кредитному портфелю [3].

Кредитный риск включает несколько аспектов, таких как дефолт заемщика (невозврат кредита), просрочка платежей или недобросовестное поведение заемщика. Оценка и управление кредитным риском являются критическими задачами для банков, поскольку неправильное принятие решений может привести к финансовым потерям и негативным последствиям для банковской организации. Кредитор оценивает кредитоспособность, анализируя информацию, предоставленную заемщиком для получения кредита [4]. Оценка кредитного риска является одним из основных инструментов, которые используются финансовыми организациями при выдаче кредитов. Скоринговые методы в банковских системах являются важной частью процесса кредитования и управления рисками.

Скоринг стал использоваться не только при оценке кредитных рисков, но

и в маркетинге для определения вероятности покупки тех или иных продуктов теми или иными покупателями. При работе с должниками, если клиент задерживается с очередным платежом, скоринг позволяет выявить, какой способ воздействия будет наиболее эффективным. Также эти методы используются для выявления мошенничества с кредитными картами и для оценки вероятности перехода клиента к конкуренту и в других подобных ситуациях [5].

Кредитный скоринг представляет собой статистический метод оценки вероятности того, что заявитель на кредит, действующий заемщик или контрагент допустит дефолт либо просрочку платежа. В рамках скорингового подхода сведения о заемщике преобразуются в числовую оценку, которая используется для ранжирования клиентов по уровню кредитного риска и поддержки решения о предоставлении кредита. Таким образом, скоринговая модель позволяет формализовать процесс оценки кредитоспособности и сделать процедуру принятия кредитного решения более объективной [6].

Наиболее используемые кредитные скоринговые системы:

- скоринг заявлений (англ. Application scoring);
- поведенческий скоринг (англ. Behavioral scoring);
- коллекторский скоринг (англ. Collection scoring);
- противомошеннический скоринг (англ. Fraud scoring).

Скоринг заявлений – это определение кредитоспособности (уровня риска дефолта) заявителя при принятии решения о предоставлении кредита на основании данных, доступных в момент подачи заявления. Фактически это анализ анкетных данных заемщика и вычисление рисков невозврата кредита. При этом принимается не только решение о предоставлении кредита, но и о размере и условиях кредитования. Основная цель при внедрении – экспресс-оценка заемщика и сокращение времени на изучение документов [7].

Коллекторский скоринг – это решение ключевых задач коллекторской деятельности, а именно планирование и осуществление своевременных и целенаправленных действий по управлению взаимоотношениями с должниками начиная с момента первого возникновения просрочки. Эффективный процесс своевременного предупреждения просрочек является очень важным для снижения

затрат по работе с долгами и залоговым имуществом. Внедрение в банке системы коллекторского скоринга преследует цель максимально использовать все возможности по предотвращению перехода клиентов из разряда «просрочек по кредиту» в «безнадежные должники» [8].

Скоринг на выявление мошенничества применяется для выявления заявок, по которым существует повышенная вероятность предоставления заемщиком недостоверной информации или совершения мошеннических действий. В кредитном процессе такие модели используются как дополнительный инструмент проверки клиента до принятия окончательного решения о выдаче кредита. На практике fraud scoring может сочетаться с экспертными правилами, проверочными фильтрами и списками ограничений, что позволяет снизить вероятность одобрения заведомо рискованных или недобросовестных заявок [9].

Общая блок-схема кредитного конвейера, на которой отображены основные этапы анализа заявки на кредит представлен на рисунке 1.1:

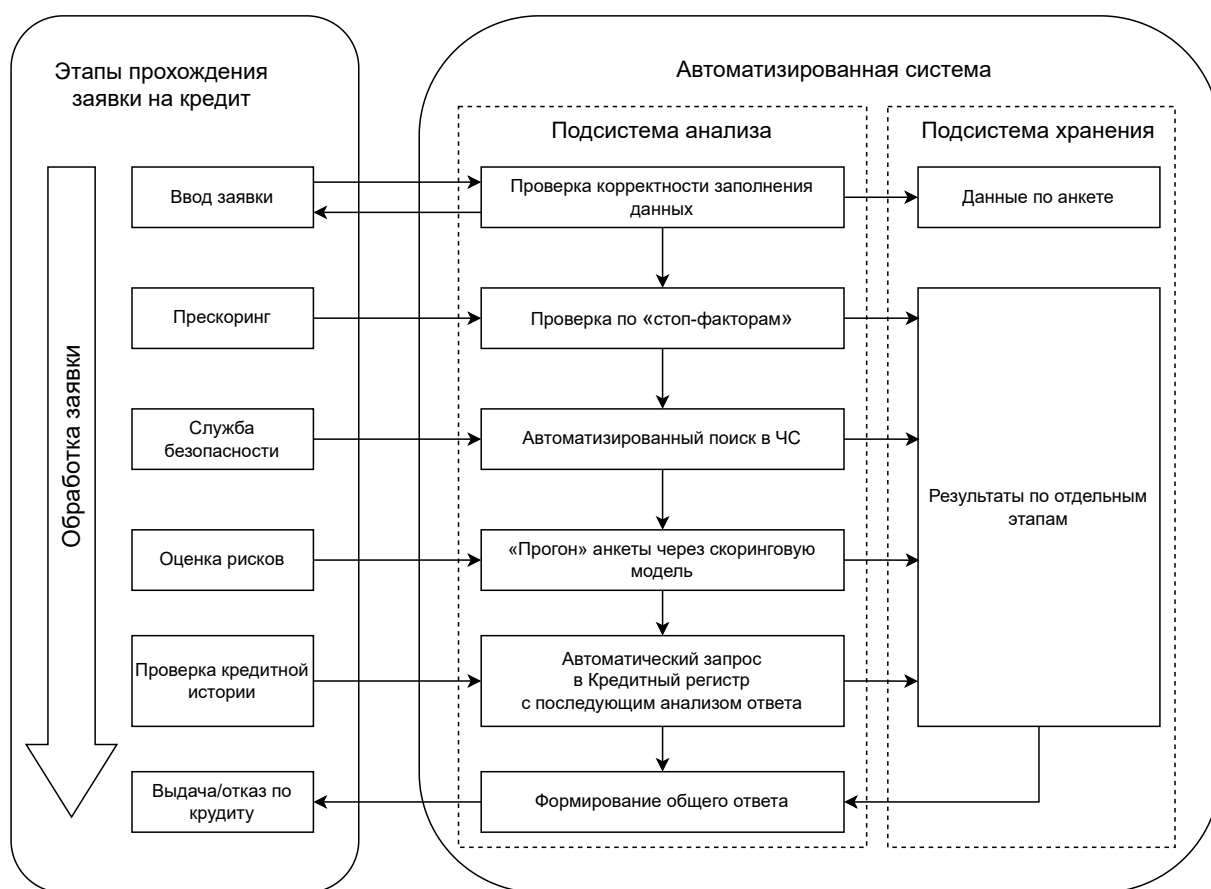


Рисунок 1.1 – Общая схема бизнес-процесса принятия кредитного решения

Скоринговые системы, особенно в условиях использования больших данных, должны соответствовать ряду требований, чтобы эффективно выполнять свои функции. Архитектура системы должна быть масштабируемой, чтобы увеличивать ее мощность по мере роста данных и требований. Также требуется возможность бесшовной интеграции с существующими банковскими системами, базами данных и IT-инфраструктурой, а также обеспечение защиты данных от несанкционированного доступа, утечек и кибератак.

Функциональные требования предусматривают применение скоринговых моделей для оценки кредитного риска с минимальной ошибкой. Важным условием является способность системы быстро обновлять модели и алгоритмы в ответ на изменения рынка и поведения клиентов.

1.2 Градиентный бустинг деревьев решений

Градиентный бустинг – это техника машинного обучения для задач классификации и регрессии, которая строит модель предсказания в форме ансамбля слабых предсказывающих моделей. При этом обучение ансамбля проводится последовательно.

Градиентный бустинг с деревьями решений в качестве базовых моделей называется GBDT. Он способен эффективно находить нелинейные зависимости в данных различной природы. Этим свойством обладают все алгоритмы, использующие деревья решений [10].

Постановка задачи

Рассмотрим задачу обучения с учителем, где имеется обучающая выборка из n наблюдений:

$$D = \{(x_i, y_i)\}_{i=1}^n, \quad (1.1)$$

где

- $x_i \in \mathbb{R}^d$ — вектор признаков;
- $y_i \in \mathbb{R}$ — целевая переменная;
- n — количество наблюдений в обучающей выборке;
- d — количество признаков.

Цель состоит в построении модели $F(x)$, которая предсказывает значение y для нового объекта x .

Общая идея градиентного бустинга

Градиентный бустинг строит модель в виде ансамбля базовых моделей:

$$F(x) = F_0(x) + \sum_{m=1}^M \gamma_m h_m(x), \quad (1.2)$$

где

- $F_0(x)$ — начальное приближение модели;
- M — количество базовых моделей;
- m — номер базовой модели;
- γ_m — коэффициент шага на m -й итерации;
- $h_m(x)$ — базовая модель на m -й итерации.

Выбор функции потерь

Выбирается дифференцируемая функция потерь, которая измеряет расхождение между истинными значениями и предсказаниями модели. Для задачи классификации может использоваться логистическая функция потерь:

$$L(y, F(x)) = \ln \left(1 + e^{-2yF(x)} \right), \quad y \in \{-1, +1\}. \quad (1.3)$$

Алгоритм градиентного бустинга

Шаг 1. Устанавливается начальное приближение модели таким образом, чтобы минимизировать функцию потерь на обучающей выборке:

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma), \quad (1.4)$$

где

- γ — постоянное значение, подбираемое при минимизации функции потерь;
- i — номер наблюдения в обучающей выборке.

Для логистической функции потерь начальное приближение может быть задано следующим образом:

$$F_0(x) = \frac{1}{2} \ln \left(\frac{p}{1-p} \right), \quad p = \frac{1}{n} \sum_{i=1}^n y_i, \quad (1.5)$$

где p — среднее значение целевой переменной в обучающей выборке.

Шаг 2. Для каждой итерации от 1 до M выполняются следующие действия.

а) Вычисляются псевдоостатки. Псевдоостатки определяются как отрицательный градиент функции потерь по предсказанию текущей модели:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}, \quad (1.6)$$

где $F_{m-1}(x)$ — модель, полученная на предыдущей итерации.

б) Базовая модель обучается на данных, где псевдоостатки используются как целевая переменная:

$$h_m(x) \approx r_{im}. \quad (1.7)$$

в) Вычисляется коэффициент шага. Оптимальное значение коэффициента шага находится путем минимизации функции потерь:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)). \quad (1.8)$$

г) Обновляется текущее приближение модели:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x). \quad (1.9)$$

Шаг 3. После выполнения M итераций получается итоговая модель:

$$F_M(x) = F_0(x) + \sum_{m=1}^M \gamma_m h_m(x). \quad (1.10)$$

Градиентный бустинг является гибким ансамблевым методом, в котором итоговая модель строится последовательно за счет добавления базовых алгоритмов, уменьшающих значение выбранной функции потерь. Такой подход позволяет применять бустинг как в задачах регрессии, так и в задачах классификации,

поскольку алгоритм может быть адаптирован к различным дифференцируемым функциям потерь. Дополнительное использование малой скорости обучения и других приемов регуляризации позволяет повысить обобщающую способность модели и снизить риск переобучения [11].

А также, модель остается интерпретируемой, так как вес каждого дерева и важность признаков можно анализировать для получения полезной информации [12].

Однако у алгоритма есть и свои недостатки. Одной из главных проблем является длительное время обучения, поскольку обучение каждого дерева выполняется последовательно, что замедляет работу на больших наборах данных. Кроме того, градиентный бустинг чувствителен к выбору гиперпараметров, таких как глубина деревьев или скорость обучения (γ) что требует тщательной настройки. Еще одним недостатком является склонность к переобучению, если не использовать регуляризацию, например, ограничение глубины деревьев или мини-батчи. Наконец, модель плохо переносит значительные изменения в тренировочной выборке, так как в таких случаях ее приходится обучать с нуля.

Для практического применения градиентного бустинга на больших наборах данных используются оптимизированные реализации, такие как XGBoost и LightGBM. XGBoost представляет собой масштабируемую систему градиентного бустинга над деревьями решений, в которой применяются алгоритмы обработки разреженных данных, приближенного поиска разбиений, а также оптимизации доступа к памяти и распределенных вычислений [13]. LightGBM, в свою очередь, ориентирован на повышение скорости обучения и снижение потребления памяти за счет использования методов Gradient-based One-Side Sampling и Exclusive Feature Bundling, что делает данный алгоритм эффективным при работе с большими объемами данных [14].

1.3 Логистическая регрессия

Логистическая регрессия применяется в задачах, где зависимая переменная имеет бинарный характер и принимает одно из двух возможных значений. В отличие от обычной линейной регрессии, данный метод используется не для непосредственного прогнозирования числового значения признака, а для оценки вероятности наступления определенного события. В модели логистической

регрессии вероятность события преобразуется через шансы и логиты, что позволяет представить связь между зависимой переменной и набором независимых признаков в виде линейной комбинации предикторов [15].

В банковских скоринговых системах логистическая регрессия используется для оценки вероятности дефолта, ранжирования клиентов по риску или интерпретации влияния признаков на вероятность дефолта.

Логистическая регрессия моделирует вероятность события $y = 1$ в зависимости от вектора признаков x с использованием логистической функции (сигмоиды):

$$P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (1.11)$$

где

- $z = w^T x + b$ — линейная комбинация признаков;
- w — вектор весов модели;
- b — смещение.

Цель обучения логистической регрессии – найти такие параметры w и b которые минимизируют функцию потерь и обеспечивают наилучшее соответствие между предсказанными вероятностями и фактическими метками классов.

Математическое описание

Для набора данных из n наблюдений $\{(x_i, y_i)\}_{i=1}^n$ функция правдоподобия модели выражается следующим образом:

$$L(w, b) = \prod_{i=1}^n P(y_i \mid x_i) = \prod_{i=1}^n [\sigma(w^T x_i + b)]^{y_i} [1 - \sigma(w^T x_i + b)]^{1-y_i}, \quad (1.12)$$

где

- $P(y_i \mid x_i)$ — условная вероятность принадлежности объекта x_i к классу y_i ;
- σ — сигмоидная функция;
- w — вектор весов модели;

— b — смещение модели.

Для удобства оптимизации используется отрицательный логарифм функции правдоподобия, известный как логистическая функция потерь:

$$J(w, b) = - \sum_{i=1}^n [y_i \ln \sigma(w^T x_i + b) + (1 - y_i) \ln (1 - \sigma(w^T x_i + b))], \quad (1.13)$$

где $J(w, b)$ — логистическая функция потерь.

Параметры модели обновляются с помощью градиентного спуска путем минимизации функции потерь:

$$w := w - \eta \frac{\partial J(w, b)}{\partial w}, \quad b := b - \eta \frac{\partial J(w, b)}{\partial b}, \quad (1.14)$$

где η — скорость обучения.

Для нового объекта x оценивается вероятность $P(y = 1 \mid x)$. Если вероятность превышает заданный порог, объект классифицируется как положительный класс:

$$\hat{y} = \begin{cases} 1, & \text{если } P(y = 1 \mid x) \geq 0,5, \\ 0, & \text{иначе,} \end{cases} \quad (1.15)$$

где $\hat{y}$ — предсказанная метка класса.

Ключевыми преимуществами логистической регрессии являются простота и интерпретируемость. Модель легко объяснима, что особенно важно для финансовых организаций и для соответствия нормативным требованиям. Кроме того, логистическая регрессия характеризуется эффективностью вычислений; обучение модели происходит относительно быстро, даже на больших наборах данных. Также при правильной регуляризации модель демонстрирует устойчивость к переобучению, будучи менее подверженной этому риску по сравнению с более сложными алгоритмами [16].

Логистическая регрессия имеет и некоторые ограничения. Одно из них — линейность модели, которая предполагает линейную зависимость между признаками и логарифмом шансов. Это может быть недостаточно для моделирования сложных нелинейных зависимостей. Модель также чувствительна к выбросам и масштабированию признаков, поэтому требуется предварительная об-

работка данных, включая нормализацию и обработку выбросов. Ограниченная выразительная способность логистической регрессии может привести к тому, что она будет уступать более сложным моделям, таким как градиентный бустинг, в точности предсказаний на сложных данных.

1.4 Линейный дискриминантный анализ

Линейный дискриминантный анализ относится к методам многомерной классификации и применяется в случаях, когда классы объектов заранее известны. Основная задача метода заключается в построении дискриминантной функции, позволяющей отнести новый объект к одному из существующих классов на основании значений его признаков. В линейном случае такая функция задается как линейная комбинация исходных показателей, параметры выбираются так, чтобы обеспечить наилучшее разделение рассматриваемых групп [17]. Скоринговый балл рассчитывается как линейная функция от значений атрибутов клиента:

$$Z = \beta^T x = \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k, \quad (1.16)$$

где

- $x = (x_1, \dots, x_k)$ — вектор значений атрибутов клиента;
- $x_1, x_2, \dots, x_k$ — отдельные атрибуты клиента;
- $\beta = (\beta_1, \dots, \beta_k)$ — вектор параметров модели;
- $\beta_1, \beta_2, \dots, \beta_k$ — параметры модели;
- k — количество атрибутов клиента.

Параметры модели выбираются таким образом, чтобы максимизировать отношение:

$$M = \frac{\beta^T(m_G - m_B)}{\sqrt{\beta^T \Sigma \beta}}, \quad (1.17)$$

где

- M — значение критерия разделения классов;
- m_G — вектор средних значений признаков для хороших клиентов;
- m_B — вектор средних значений признаков для плохих клиентов;
- Σ — общая ковариационная матрица.

Недостатком является то, что ЛДА требует нормально распределенных данных, но данные о кредитных операциях часто являются слабоструктурированными и не всегда есть возможность их классифицировать.

Линейный дискриминантный анализ предполагает равенство ковариационных матриц и нормальное распределение независимых переменных. В задачах кредитного скоринга эти предположения часто нарушаются, поскольку признаки могут быть дискретными или не подчиняться нормальному распределению. Это усложняет применение метода. Однако показано, что при нарушении условия данный метод остается применим на практике [18].

Схожий метод линейной регрессии, также используется для формирования скоринговой модели. В случае двух категорий, он эквивалентен методу линейного дискриминантного анализа и выражает зависимость одной переменной (зависимой) от других (независимых). В общем виде представляется так:

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n + \epsilon, \quad (1.18)$$

где

- Y — зависимая переменная;
- β_0 — свободный член модели;
- X_i — объясняющая независимая переменная;
- β_i — коэффициент регрессии при переменной X_i ;

- n — количество объясняющих переменных;
- ϵ — случайная ошибка.

Для применения модели линейного скоринга требуется выполнение следующего предположения: связь между зависимой и независимыми переменными должна быть линейной. В противном случае, точность оценки значительно ухудшается. Ошибки же должны быть независимы и распределены нормально.

Как и в случае дискриминантного анализа, в условиях кредитного скоринга, предположения, требуемые для применения линейной регрессии, нередко нарушаются. Линейная регрессия может дать оценку вероятности вне диапазона $[0,1]$, что является неприемлемым.

1.5 Метод опорных векторов

Метод опорных векторов (support vector machine, SVM) относится к методам интеллектуального анализа данных для решения задачи классификации. Его основная идея заключается в переводе исходных векторов в пространство более высокой размерности с применением метода ядра для обеспечения линейной разделимости классов и в поиске разделяющей гиперплоскости с максимальным зазором между гиперплоскостью и опорными векторами в этом пространстве [19].

Постановка задачи

Пусть имеется обучающая выборка:

$$D = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^d, \quad y_i \in \{-1, +1\}. \quad (1.19)$$

Метод SVM строит разделяющую гиперплоскость, минимизируя ошибку классификации и максимизируя зазор между классами:

$$\mathcal{H}(w, b) = \{x \in \mathbb{R}^d : w^T x + b = 0\}, \quad (1.20)$$

где w — вектор нормали к гиперплоскости.

Оптимизационная задача SVM

Для линейно разделимых данных SVM решает следующую задачу оптимизации:

$$\min_{w,b} \frac{1}{2} \|w\|^2, \quad \text{при ограничениях } y_i(w^T x_i + b) \geq 1, \quad \forall i. \quad (1.21)$$

Данная формулировка минимизирует норму вектора w , что эквивалентно максимизации зазора между классами.

Для линейно неразделимых данных вводится штраф за ошибочную классификацию с использованием мягкого зазора:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i, \quad \text{при ограничениях } y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i, \quad (1.22)$$

где C — гиперпараметр, регулирующий допустимый уровень ошибок при построении разделяющей гиперплоскости.

На рисунке 1.2 показана разница между идеальным разделением классов (hard margin) и случаем с допущением ошибок классификации (soft margin). В случае мягкого зазора некоторые точки могут находиться внутри разделяющей полосы или за ее пределами, что учитывается через переменные ξ_i .

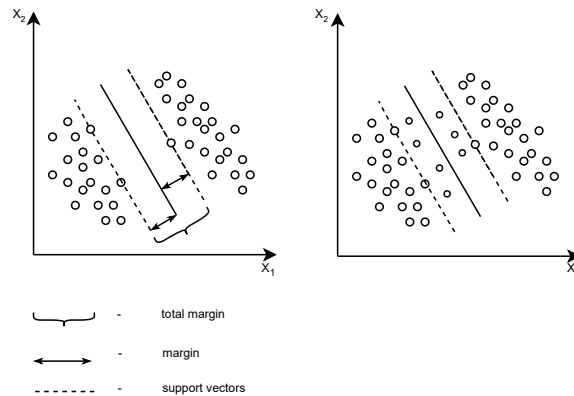


Рисунок 1.2 – Принцип работы линейного SVM с жестким и мягким зазором

Ядерный метод в SVM

Для линейно неразделимых данных в исходном пространстве SVM использует ядра, которые позволяют перенести данные в пространство более высокой размерности, где классы становятся линейно разделимыми. Скалярное

произведение в новом пространстве, неявно заданном отображением $\phi(x)$, вычисляется с помощью ядерной функции $K(x_i, x_j)$:

$$K(x_i, x_j) = \gamma(x_i)^T \gamma(x_j). \quad (1.23)$$

Решение двойственной задачи

SVM решает двойственную задачу оптимизации, что позволяет использовать ядра:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j), \quad (1.24)$$

где

- α_i — коэффициент, определяющий важность i -го объекта;
- j — номер объекта в обучающей выборке;
- $K(x_i, x_j)$ — ядерная функция, определяющая меру сходства объектов x_i и x_j .

При этом выполняются следующие ограничения:

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0. \quad (1.25)$$

Только объекты, лежащие на границах зазора или внутри него, то есть опорные векторы, имеют ненулевые значения α_i и влияют на итоговую модель.

Метод опорных векторов (SVM) находит широкое применение в кредитном скоринге, где требуется оценить вероятность выполнения клиентом определенных обязательств. SVM решает задачу бинарной классификации, разделяя клиентов на платежеспособных и неплатежеспособных.

Скоринг формулируется как задача бинарной классификации: положительный класс ($y = +1$) соответствует платежеспособным клиентам, а отрицательный класс ($y = -1$) — неплатежеспособным. Входные признаки x_i включают данные о клиенте, такие как возраст, доход, кредитная история и другие характеристики. На основе обучающей выборки, содержащей исторические данные о клиентах, SVM строит оптимальную разделяющую гиперплоскость. Для повышения качества модели может использоваться мягкий зазор (soft margin), позво-

ляющий учитывать ошибки классификации. Полученный результат классификации преобразуется в числовой балл, отражающий уровень риска клиента:

$$\text{Score} = w^T x + b, \quad (1.26)$$

где

- Score — скоринговый балл, отражающий уровень риска клиента;
- x — вектор признаков клиента;
- b — смещение модели.

Положительное значение Score указывает на низкий риск дефолта, а отрицательное — на высокий.

Преимуществами метода опорных векторов (SVM) являются высокая скорость нахождения решающих функций и возможность свести задачу к решению квадратичного программирования в выпуклой области, которая всегда имеет единственное решение. Кроме того, SVM находит разделяющую гиперплоскость с максимальной шириной полосы, что позволяет осуществлять более уверенную классификацию.

Однако метод чувствителен к шумам и требует тщательной стандартизации данных. Отсутствует общий подход к выбору ядра и построению спрямляющего подпространства при работе с линейно неразделимыми классами. SVM также плохо масштабируется на большие объемы данных, так как решает задачу квадратичного программирования, сложность которой растет квадратично или кубически с увеличением размера обучающей выборки. Шумы в данных могут оказывать существенное влияние на положение разделяющей гиперплоскости, особенно при использовании мягкого зазора, так как ошибочно классифицированные объекты могут стать опорными векторами. Метод требует тщательной предварительной обработки данных, включая нормализацию или стандартизацию признаков и управление выбросами, чтобы минимизировать влияние масштаба данных и выбросов на результат модели [20].

1.6 Классификация существующих решений

В качестве основных параметров классификации алгоритмов были использованы признаки, приведенные в таблице 1:

Таблица 1 – Параметры классификации алгоритмов

	Параметр
1	Способность модели прогнозировать целевую переменную (например, вероятность дефолта клиента)
2	Тип модели
3	Параметричность
4	Масштабируемость и обработка больших данных
5	Предположения о данных
6	Устойчивость к переобучению и шуму
7	Тип алгоритма
8	Способность моделировать сложные зависимости
9	Требования к предварительной обработке данных
10	Устойчивость к выбросам
11	Время обучения и предсказания
12	Совместимость с технологиями больших данных
13	Поддержка параллельных вычислений и распределенной обработки данных

Классификация существующих алгоритмов представлена в таблице 2.

Таблица 2 – Таблица сравнения существующих алгоритмов

Критерий	Градиентный бустинг деревьев решений	Логистическая регрессия	Линейный дискриминантный анализ	Метод опорных векторов (SVM)
Способность модели точно прогнозировать целевую переменную	Высокая точность за счет сложных нелинейных зависимостей	Умеренная точность при линейных зависимостях	Точность зависит от выполнения предположений о данных	Высокая, при правильном выборе ядра
Тип модели	Нелинейная	Линейная	Линейная	Нелинейная
Параметричность	Непараметрическая	Параметрическая	Параметрическая	Непараметрическая
Обработка больших данных	Высокая (с библиотеками)	Высокая	Средняя	Низкая; плохо масштабируется на большие выборки
Предположения о данных	Минимальные	Линейность, независимость признаков	Нормальность, равенство ковариаций	Минимальные, но требуется стандартизация данных
Устойчивость к переобучению и шуму	Требует настройки гиперпараметров для предотвращения переобучения	Устойчива при использовании регуляризации	Чувствительна к выбросам и нарушениям предположений	Чувствителен к шумам, но настраиваемый параметр C помогает снизить их влияние
Тип алгоритма	Ансамблевый метод (бустинг)	Одиночная модель	Одиночная модель	Одиночная модель
Способность моделировать сложные зависимости	Высокая	Ограничена (только линейные зависимости)	Ограничена линейностью модели	Высокая, при использовании нелинейных ядер
Требования к предварительной обработке данных	Низкие; не требует масштабирования признаков	Высокие; требует масштабирования и обработки выбросов	Высокие; требует нормальности распределения и равенства ковариационных матриц	Высокие; требует масштабирования и удаления выбросов
Устойчивость к выбросам	Относительно устойчива	Чувствительна к выбросам	Очень чувствительна к выбросам	Чувствителен к выбросам
Время обучения и предсказания	Длительное обучение; быстрое предсказание	Быстрое обучение и предсказание	Обучение может быть медленным из-за вычисления ковариационных матриц	Медленное обучение; быстрое предсказание

Продолжение таблицы 2

Критерий	Градиентный бустинг деревьев решений	Логистическая регрессия	Линейный дискриминантный анализ	Метод опорных векторов (SVM)
Совместимость с технологиями больших данных	Да; поддерживается оптимизированными библиотеками (XGBoost, LightGBM)	Да; легко интегрируется	Ограничена; требует дополнительных усилий для интеграции	Ограничена; требует модификаций для больших данных
Поддержка параллельных вычислений и распределенной обработки данных	Да; оптимизированные библиотеки используют многоядерные и распределенные вычисления	Ограничена; в базовой реализации требуется настройка	Ограничена; требует существенных усилий для распараллеливания	Ограничена; базовые реализации не поддерживают распараллеливание

1.7 Скоринговая система FICO

FICO Score является одной из наиболее распространенных скоринговых систем, используемых кредиторами для оценки кредитного риска заемщиков. Данный показатель рассчитывается на основе информации, содержащейся в кредитной истории клиента, и обычно находится в диапазоне от 300 до 850 баллов. Более высокое значение FICO Score соответствует более низкому уровню риска для кредитора и может положительно влиять на вероятность одобрения кредита и условия его предоставления [21].

Расчет FICO Score основывается на анализе кредитной истории заемщика и включает несколько ключевых факторов:

- платежная дисциплина (около 35%) – своевременность погашения задолженностей;
- уровень задолженности (около 30%) – отношение текущих обязательств к доступному кредитному лимиту;
- длительность кредитной истории (около 15%);
- структура кредитов (около 10%) – разнообразие кредитных продуктов;
- новые кредитные заявки (около 10%).

Модель FICO относится к классу проприетарных решений, что означает отсутствие открытой информации о точных алгоритмах расчета. Однако известно, что в основе системы лежат статистические методы анализа данных, включая элементы логистической регрессии и скоринговых карт.

К преимуществам системы FICO можно отнести широкое распространение, стандартизацию оценки кредитного риска и высокую интерпретируемость результатов. Вместе с тем, использование фиксированной модели ограничивает ее адаптивность к изменениям в поведении заемщиков и особенностям локальных рынков.

Система FICO представляет собой классический пример промышленного скорингового решения, на основе которого развиваются современные методы оценки кредитоспособности, включая модели машинного обучения.

1.8 Скоринговые решения Experian

Решения Experian в области кредитного скоринга и принятия решений ориентированы на автоматизацию кредитного процесса и управление рисками на различных этапах жизненного цикла клиента. В качестве одного из таких решений используется платформа PowerCurve, предназначенная для объединения данных, аналитики и технологий принятия решений в единой системе. Данная платформа позволяет автоматизировать кредитные решения, использовать аналитические модели и учитывать информацию, связанную с кредитным риском, проверкой клиента и противодействием мошенничеству.

Современные решения Experian используют не только традиционные данные кредитной истории, но и расширенные источники информации, а также методы аналитики и машинного обучения. Это позволяет преобразовывать сложные данные в прикладные решения для оценки риска, управления портфелем, привлечения клиентов и сопровождения кредитного процесса.

К основным возможностям решений Experian относятся:

- построение и настройка скоринговых моделей под конкретные бизнес-задачи;
- использование как традиционных, так и альтернативных источников данных;

- интеграция с внутренними банковскими системами и внешними сервисами;
- автоматизация процесса принятия решений в кредитном конвейере;
- мониторинг и управление кредитным риском в режиме реального времени;
- применение методов машинного обучения для повышения качества прогнозирования.

Преимуществом решений Experian является их высокая адаптивность и возможность учитывать изменения в поведении клиентов и рыночной среде. Использование больших данных и методов машинного обучения позволяет существенно повысить точность скоринговых моделей по сравнению с традиционными подходами.

Внедрение подобных систем связано с рядом сложностей, включая необходимость обработки больших объемов данных, обеспечение их качества, а также наличие вычислительных ресурсов и квалифицированных специалистов.

Таким образом, решения Experian представляют собой современные скоринговые системы, использующие методы анализа больших данных и машинного обучения, что делает их более гибкими и эффективными по сравнению с классическими скоринговыми моделями.

1.9 Скоринговые системы на основе машинного обучения

Современное развитие кредитного скоринга связано с активным применением методов машинного обучения для автоматизированной оценки кредитных рисков. Такие методы позволяют анализировать большие объемы клиентских данных, выявлять сложные зависимости между признаками заемщика и вероятностью дефолта, а также повышать качество прогнозирования по сравнению с рядом традиционных статистических подходов.

В задачах оценки кредитного риска применяются различные алгоритмы машинного обучения, включая логистическую регрессию, метод k-ближайших соседей, деревья решений, случайный лес и градиентный бустинг. Особое зна-

чение имеют ансамблевые методы, так как они позволяют объединять несколько моделей и повышать устойчивость итогового прогноза [22].

Наиболее распространенными инструментами для построения таких моделей являются библиотеки и платформы анализа данных, такие как:

- Scikit-learn – библиотека машинного обучения на языке Python;
- XGBoost – высокоэффективная реализация градиентного бустинга;
- LightGBM – оптимизированный алгоритм бустинга для работы с большими данными;
- CatBoost – алгоритм градиентного бустинга, эффективно работающий с категориальными признаками.

Использование методов машинного обучения позволяет учитывать сложные нелинейные зависимости между признаками, а также эффективно работать с большими объемами данных. Это приводит к повышению точности оценки кредитного риска по сравнению с традиционными подходами.

Вместе с тем, такие модели обладают меньшей интерпретируемостью, что может затруднять их использование в условиях строгого регулирования финансового сектора. В связи с этим на практике часто применяются гибридные подходы, сочетающие интерпретируемые модели и алгоритмы машинного обучения.

Таким образом, использование методов машинного обучения является современным направлением развития скоринговых систем и представляет собой перспективную основу для построения моделей оценки кредитоспособности.

1.10 Формулировка цели

В ходе выполнения работы требуется разработать модель, позволяющая оценивать кредитоспособность физических лиц в банковских системах с использованием алгоритма градиентного бустинга деревьев решений. Разрабатываемая модель должна отвечать следующим требованиям:

1. Обеспечивать высокую точность прогнозирования вероятности дефолта заемщика.

2. Учитывать совокупность финансовых и социально-демографических характеристик клиента.
3. Обеспечивать возможность работы с большими объемами данных.
4. Обладать устойчивостью к шумам и выбросам в данных.
5. Обеспечивать приемлемое время обучения и предсказания.
6. Позволять интерпретировать результаты оценки кредитного риска.

1.11 Формализация задачи. IDEF0-диаграмма

Для формализации задачи была составлена IDEF0-диаграмма нулевого и первого уровня.

IDEF0-диаграмма нулевого уровня представлена на рисунке 1.3:

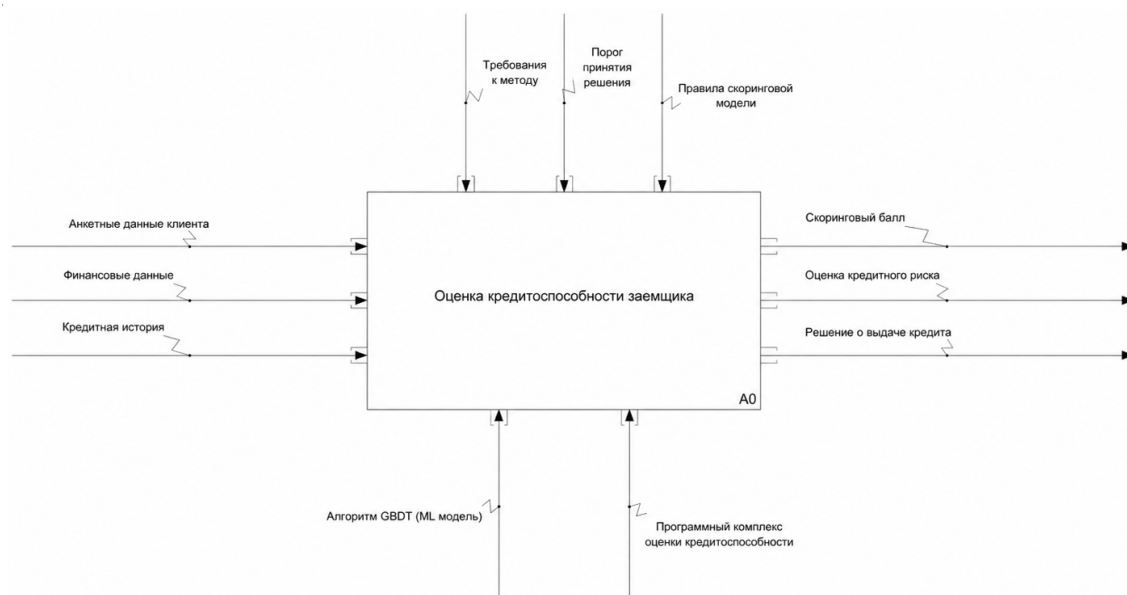


Рисунок 1.3 – IDEF0-диаграмма нулевого уровня

IDEF0-диаграмма нулевого уровня (A-0) отражает общий процесс оценки кредитоспособности заемщика. На вход системы поступают анкетные данные клиента, финансовые данные и кредитная история. Управляющими воздействиями выступают требования к методу, порог принятия решения и правила скоринговой модели. В качестве механизмов используются алгоритм GBDT (ML модель) и программный комплекс оценки кредитоспособности. Результатом вы-

полнения процесса являются скоринговый балл, оценка кредитного риска и решение о выдаче кредита.

IDEF0-диаграмма первого уровня представлена на рисунке 1.4:

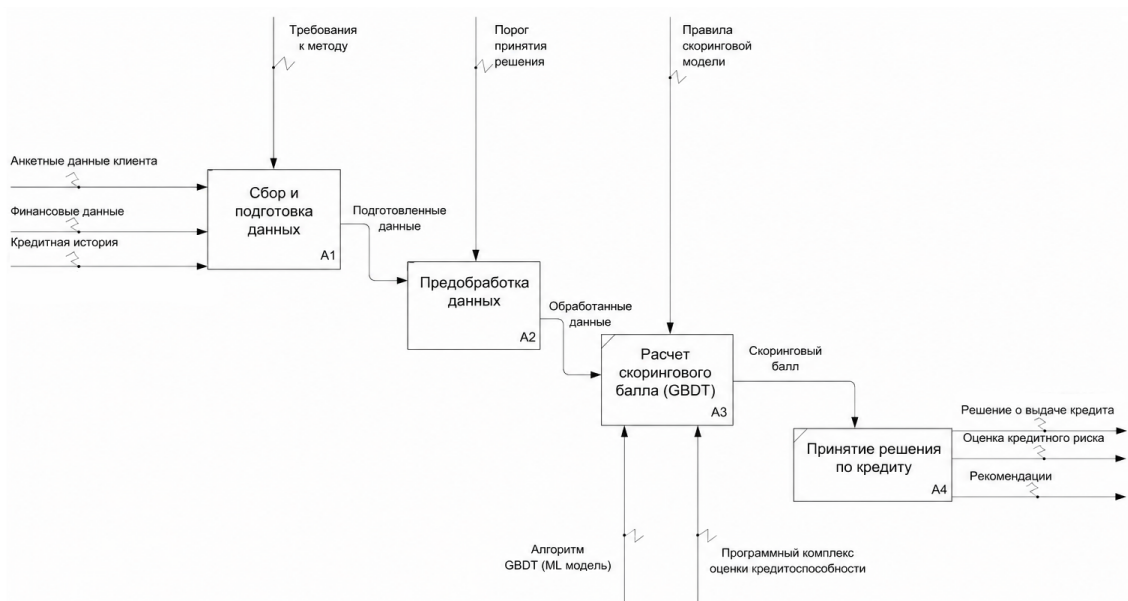


Рисунок 1.4 – IDEF0-диаграмма первого уровня

IDEF0-диаграмма первого уровня (A0) детализирует процесс оценки кредитоспособности заемщика и разбивает его на последовательные этапы.

На этапе A1 выполняется сбор и подготовка данных. На вход данного этапа поступают анкетные данные клиента, финансовые данные и кредитная история. Результатом этапа являются подготовленные данные.

На этапе A2 осуществляется предобработка данных. Подготовленные данные проходят обработку и преобразуются в обработанные данные, пригодные для дальнейшего расчета.

На этапе A3 выполняется расчет скорингового балла (GBDT) с использованием алгоритма GBDT (ML модель) и программного комплекса оценки кредитоспособности. На данный этап также воздействуют правила скоринговой модели. Результатом этапа является скоринговый балл.

На этапе A4 выполняется принятие решения по кредиту. На основе скорингового балла, порога принятия решения и требований к методу формируются итоговые результаты: решение о выдаче кредита, оценка кредитного риска и рекомендации.

Таким образом, декомпозиция позволяет выделить основные этапы обработки данных, расчета скорингового балла и принятия кредитного решения, а

также показать взаимосвязь входных данных, управляющих воздействий, механизмов и результатов.

Выводы

Был проведен анализ предметной области оценки кредитоспособности физических лиц, введены основные понятия кредитного скоринга. Рассмотрены методы машинного обучения, применяемые для оценки кредитного риска заемщиков, включая алгоритм градиентного бустинга деревьев решений.

Проанализированы традиционные статистические методы, такие как логистическая регрессия и линейный дискриминантный анализ, а также современные алгоритмы машинного обучения, включая метод опорных векторов и градиентный бустинг. Результаты сравнительного анализа представлены в табличном виде.

Была сформулирована цель работы. Постановка задачи формализована в виде IDEF0-диаграммы.

2 Конструкторский раздел

В данном разделе формулируются требования к разрабатываемому методу оценки кредитоспособности заемщика. Приводится описание метода, основанного на алгоритме градиентного бустинга деревьев решений, а также определяются его основные характеристики и область применения в задаче кредитного скоринга.

Рассматриваются этапы работы метода, включающие предобработку исходных данных, обучение модели и получение предсказаний кредитного риска. Описывается алгоритм реализации метода, включая последовательность вычислительных процедур и используемые математические модели. Раскрываются принципы интерпретации результатов моделирования.

Приводится описание структур данных, применяемых в системе, а также их назначение в процессе обработки информации. Определяется структура программного комплекса и описывается взаимодействие его компонентов, обеспечивающих реализацию разработанного метода.

2.1 Требования к разрабатываемому методу

Метод оценки кредитоспособности заемщика должен удовлетворять следующим требованиям:

1. Принимать на вход данные о заемщике, представленные в виде набора признаков.
2. Обеспечивать предварительную обработку исходных данных, включающую преобразование признаков и обработку пропущенных значений.
3. Осуществлять обучение модели на основе алгоритма градиентного бустинга деревьев решений.
4. Формировать предсказание кредитоспособности заемщика в виде бинарной классификации.
5. Обеспечивать возможность интерпретации результатов работы модели, включая определение значимости признаков.

6. Обеспечивать достаточную точность прогнозирования для применения в задачах кредитного скоринга.

2.2 Требования к программному комплексу

Программный комплекс, реализующий интерфейс к разработанному методу оценки кредитоспособности заемщика, должен обеспечивать выполнение следующих функций:

- загрузку исходных данных о заемщиках с использованием пользовательского интерфейса;
- выполнение предварительной обработки данных, включающей преобразование признаков и подготовку данных к обучению модели;
- запуск процесса обучения модели на основе алгоритма градиентного бустинга деревьев решений;
- выполнение предсказания кредитоспособности заемщика на основе обученной модели;
- отображение результатов работы метода, включая значение скорингового балла и оценку кредитного риска;
- отображение значимости признаков, используемых моделью, для интерпретации результатов.

Входные данные загружаются из файловой системы в формате JSON. Система поддерживает повторный запуск скоринга после изменения входных данных или параметров модели. При повторном запуске формируется новый входной JSON-файл, который передается в модуль предсказания.

2.3 Основной алгоритм работы

Учитывая особенности существующих методов оценки кредитоспособности, рассмотренные ранее, был разработан метод, основанный на алгоритме градиентного бустинга деревьев решений.

На первом этапе осуществляется загрузка и подготовка исходных данных о заемщиках. Данные приводятся к требуемому формату, выполняется обработка пропущенных значений, преобразование признаков и разделение данных на обучающую и тестовую выборки, необходимых для последующего обучения модели.

Далее производится обучение модели на основе подготовленных данных. В процессе обучения формируется ансамбль решающих деревьев, позволяющий учитывать сложные зависимости между признаками и повышать точность прогнозирования. Обучение продолжается до достижения заданного критерия качества модели.

После завершения обучения модель сохраняется и в дальнейшем используется для формирования предсказаний. Для этого выполняется загрузка обученной модели и обработка новых данных о заемщике, прошедших этап предобработки.

Модель по входным данным выполняет бинарную классификацию и относит заемщика к одному из классов: кредитоспособен или некредитоспособен. Результат работы модели может быть дополнительно проанализирован с использованием методов интерпретации, включая оценку значимости признаков. Такой анализ позволяет определить, какие характеристики заемщика оказывают наибольшее влияние на итоговое предсказание модели, и повышает прозрачность применения алгоритмов машинного обучения в задаче кредитного скоринга [23].

Схема алгоритма представлена на рисунке 2.1.

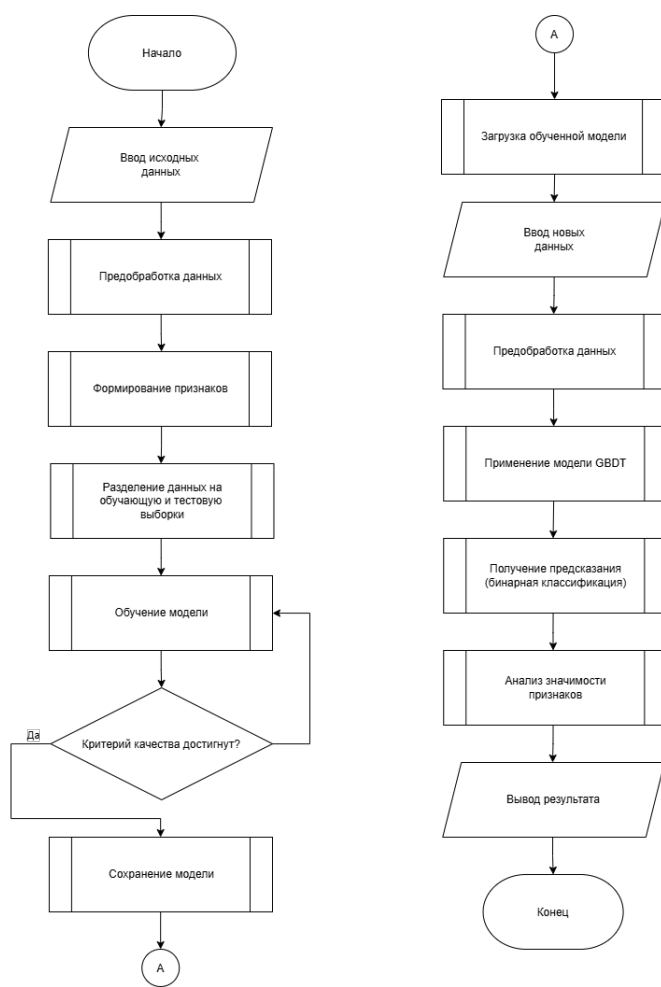


Рисунок 2.1 – Верхнеуровневая схема алгоритма

2.4 Описание входных данных и признакового пространства

Входные данные для разработанного метода оценки кредитоспособности представляют собой совокупность характеристик физического лица, отражающих его финансовое состояние, социально-демографические особенности и кредитную историю.

Каждое физическое лицо описывается вектором признаков следующего вида:

$$x = (x_1, x_2, \dots, x_d), \quad (2.1)$$

где

— x — вектор признаков объекта;

— x_i — значение i -го признака;

— d — общее количество признаков.

В рамках разработанного метода используется набор признаков, включающий следующие группы:

- **Социально-демографические характеристики:** возраст, семейное положение, уровень образования и другие параметры заемщика;
- **Финансовые характеристики:** уровень дохода, параметры запрашиваемого кредита, размер платежей и долговая нагрузка;
- **Параметры занятости:** тип занятости, стаж работы и характеристики источника дохода;
- **Кредитная история:** сведения о ранее полученных кредитах, наличии задолженностей и просрочек;
- **Агрегированные поведенческие признаки:** показатели, полученные на основе истории предыдущих заявок и фактического выполнения платежей по кредитам.

Признаки характеризуются различным типом данных: они могут быть количественными (например, уровень дохода, величина расходов) и категориальными (например, тип занятости, уровень образования). В связи с этим на этапе предобработки требуется их дополнительное преобразование.

Целевая переменная задачи определяется как бинарный признак:

$$y \in \{0, 1\}, \quad (2.2)$$

где

- y — метка класса;
- $y = 1$ — заемщик с низким риском дефолта (кредитоспособный);
- $y = 0$ — заемщик с высоким риском дефолта (некредитоспособный).

Таким образом, задача оценки кредитоспособности формализуется как задача бинарной классификации, в которой по вектору признаков x необходимо определить значение целевой переменной y .

2.5 Предобработка данных

Для обеспечения корректной работы модели и повышения качества прогнозирования выполняется этап предобработки исходных данных. В задачах прогнозного моделирования предобработка является одним из ключевых этапов, поскольку исходные данные могут содержать пропущенные значения, выбросы, признаки различного масштаба и категориальные переменные. Их предварительное преобразование позволяет подготовить данные к обучению модели и снизить влияние некорректных или неполных наблюдений на итоговое качество прогнозирования [24].

В разработанном методе предобработка состоит из следующих этапов:

- обработка пропущенных значений предполагает заполнение пропусков медианными значениями для числовых признаков и часто встречающимися значениями для категориальных признаков, что позволяет снизить влияние неполноты данных на качество модели;
- обработка выбросов выполняется путем ограничения значений признаков по заданным порогам (квантильное усечение), что повышает устойчивость модели к экстремальным значениям доходов и других финансовых характеристик заемщика;
- кодирование категориальных признаков осуществляется с использованием метода one-hot кодирования, что обеспечивает корректную обработку категориальных данных алгоритмом машинного обучения;
- масштабирование признаков при необходимости используется для приведения числовых признаков к сопоставимому диапазону значений, однако для алгоритма градиентного бустинга данный этап не является обязательным;
- формирование производных признаков включает расчет дополнительных показателей, отражающих финансовое состояние заемщика, таких как:
 - отношение ежемесячных платежей по кредитам к доходу (DTI);
 - отношение расходов к доходу;

- коэффициент долговой нагрузки;

После выполнения предобработки формируется итоговый набор признаков, используемый для обучения модели и построения прогнозов кредитоспособности.

2.6 Параметры и настройка модели

В рамках разработанного метода для оценки кредитоспособности используется модель градиентного бустинга деревьев решений. Качество такой модели во многом определяется выбором гиперпараметров, включая количество деревьев, глубину деревьев, скорость обучения и параметры регуляризации. Эти параметры влияют на сложность ансамбля, скорость обучения и способность модели обобщать закономерности на новых данных [11].

К основным параметрам модели относятся:

- количество деревьев M определяет размер ансамбля; увеличение числа деревьев позволяет повысить точность модели, однако приводит к увеличению времени обучения и риску переобучения;
- глубина деревьев ограничивает сложность базовых алгоритмов; небольшая глубина способствует лучшей обобщающей способности модели, тогда как чрезмерная глубина может привести к переобучению;
- скорость обучения γ определяет вклад каждого дерева в итоговую модель; малые значения обеспечивают более стабильное обучение, но требуют увеличения числа итераций;
- минимальное количество объектов в листе дает возможность контролировать процесс разбиения деревьев и снижает влияние шумовых данных;
- параметры регуляризации используются для предотвращения переобучения модели и повышения ее обобщающей способности;

Настройка гиперпараметров осуществляется на основе обучающей выборки с использованием процедуры валидации. Разделение исходных данных

на обучающую и тестовую выборки позволяет сначала построить модель на одной части данных, а затем оценить ее качество на независимых наблюдениях, которые не использовались при обучении [25].

В процессе настройки подбираются такие значения параметров, при которых достигается наилучшее качество классификации при сохранении устойчивости модели к переобучению.

2.7 Формирование результата и принятие решения

После завершения обучения модель градиентного бустинга используется для оценки кредитоспособности физического лица на основе входных данных.

На вход модели подается вектор признаков заемщика x , прошедший этап предобработки. В результате модель формирует числовую оценку, которая интерпретируется как вероятность наступления события дефолта:

$$P(y = 1 \mid x), \quad (2.3)$$

где $P(y = 1 \mid x)$ — вероятность того, что заемщик с признаками x относится к классу кредитоспособных заемщиков.

Для удобства интерпретации полученная вероятность может быть преобразована в скоринговый балл, отражающий уровень кредитного риска заемщика.

Принятие решения о предоставлении кредита осуществляется на основе порогового значения вероятности. Введем порог T , определяемый кредитной политикой банка.

Решение принимается следующим образом:

$$\hat{y} = \begin{cases} 1, & \text{если } P(y = 1 \mid x) \geq T, \\ 0, & \text{если } P(y = 1 \mid x) < T. \end{cases} \quad (2.4)$$

Таким образом, если вероятность кредитоспособности превышает установленный порог, заемщик относится к категории надежных и принимается положительное решение о выдаче кредита. В противном случае заемщик классифицируется как рискованный, и в выдаче кредита может быть отказано.

Пороговое значение T может варьироваться в зависимости от кредитной политики банка, уровня допустимого риска и текущей экономической ситуации.

Дополнительно результат работы модели может использоваться для ранжирования заемщиков по уровню риска и определения индивидуальных условий кредитования, таких как процентная ставка или размер кредитного лимита.

2.8 Структуры данных

При реализации метода оценки кредитоспособности применяются структуры данных для хранения, обработки и передачи информации между компонентами системы.

Основной структурой является набор данных о физических лицах, представленный в виде таблицы, где каждая строка соответствует отдельному заемщику, а столбцы — признакам, характеризующим его финансовое состояние и поведенческие характеристики.

Для представления входных данных используется следующая структура:

- идентификатор заемщика;
- значения признаков (социально-демографические параметры, кредитная история, финансовые характеристики а также агрегированные поведенческие показатели заемщика);
- целевая переменная (для обучающей выборки).

Обучающая выборка формируется в виде матрицы признаков X и вектора целевой переменной y :

$$X \in \mathbb{R}^{n \times d}, \quad y \in \{0, 1\}^n, \quad (2.5)$$

где

- X — матрица признаков;
- y — вектор целевых значений;
- n — количество объектов;

Модель градиентного бустинга представляется в виде ансамбля решающих деревьев, параметры которых включают структуру деревьев, значения весов и коэффициенты модели.

Результат работы модели представляет собой структуру данных, включающую вероятность кредитоспособности, скоринговый балл, класс заемщика, а также значения важности признаков.

Для хранения промежуточных данных используются структуры, содержащие результаты предобработки, а также параметры модели, полученные в процессе обучения.

2.9 Взаимодействие компонентов системы

Метод реализован как программный комплекс из взаимосвязанных компонентов, выполняющих полный цикл обработки данных и принятия решений. В структуре системы выделяются следующие основные модули:

- Модуль загрузки данных обеспечивает получение исходной информации о физических лицах из внешних источников или пользовательского интерфейса.
- Модуль предобработки данных выполняет очистку данных, кодирование признаков, обработку пропущенных значений и формирование итогового набора признаков.
- Модуль обучения модели реализует процесс построения модели на основе алгоритма градиентного бустинга деревьев решений с использованием обучающей выборки.
- Модуль предсказания обеспечивает применение обученной модели к новым данным и расчет вероятности кредитоспособности заемщика.
- Модуль интерпретации результатов выполняет анализ важности признаков и формирование объяснений полученных результатов.
- Модуль визуализации и вывода результатов отображает результаты работы системы, включая скоринговый балл, оценку кредитного риска и рекомендации по принятию решения.

Система работает в двух режимах: режиме предсказания (online) и режиме обучения модели (offline). В режиме обучения используются исторические данные о физических лицах. Эти данные проходят этап предобработки и поступают в модуль обучения, где формируется модель градиентного бустинга. В режиме предсказания новые данные о заемщике проходят этап предобработки и передаются в модуль предсказания, который использует ранее обученную модель для расчета вероятности кредитоспособности.

Взаимодействие компонентов осуществляется последовательно. Исходные данные поступают в модуль загрузки, после чего передаются в модуль предобработки. Подготовленные данные используются либо для обучения модели, либо для получения предсказания. Результаты работы модели поступают в модуль интерпретации, где определяется значимость признаков, а затем передаются в модуль визуализации для отображения пользователю.

На рисунке 2.2 представлена схема взаимодействия компонентов системы.

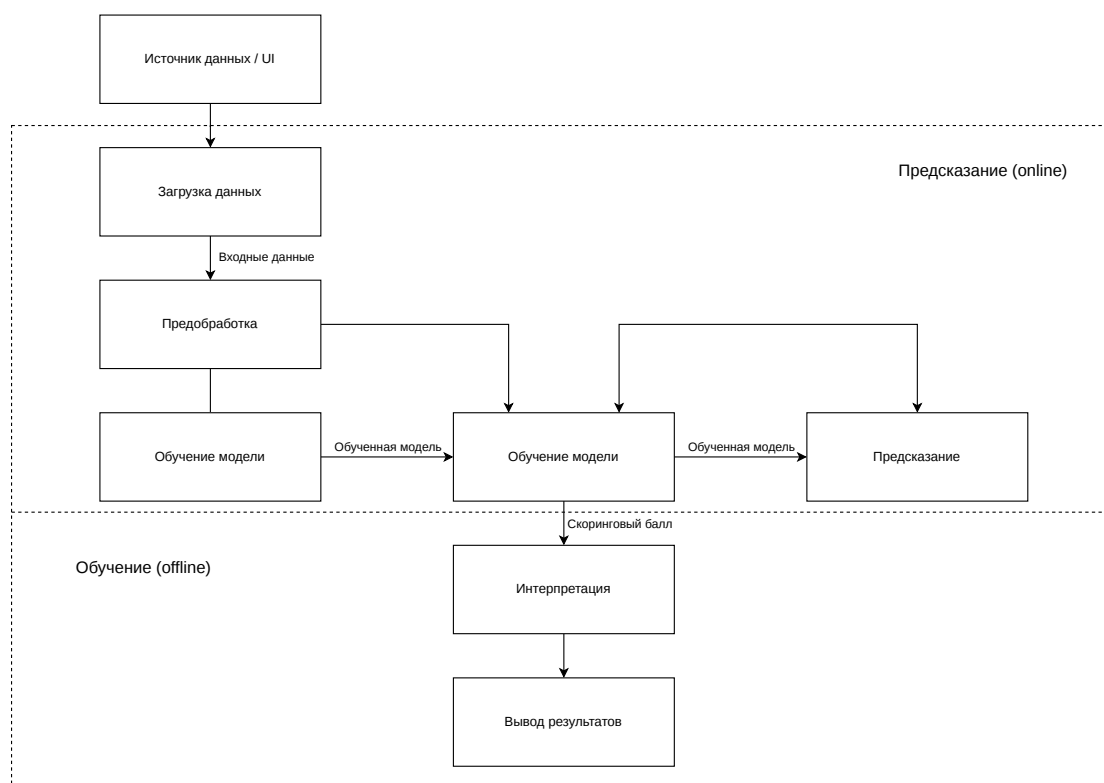


Рисунок 2.2 – Схема взаимодействия компонентов системы

Выводы

В данном разделе был разработан метод оценки кредитоспособности физических лиц на основе алгоритма градиентного бустинга деревьев решений. Были сформулированы требования к методу и программному комплексу, а также описаны основные этапы его функционирования.

Представлено описание признакового пространства, включающего финансовые характеристики, социально-демографические параметры, кредитную историю, а также поведенческие признаки заемщиков. Рассмотрены методы предобработки данных, направленные на повышение качества модели. Рассмотрены этапы обучения модели, формирования предсказаний и принятия решения на основе порога вероятности.

Также были описаны используемые структуры данных и разработана архитектура программного комплекса, включающая взаимодействие его основных компонентов. Для наглядного представления работы системы приведена структурная схема.

3 Технологический раздел

В данном разделе обосновывается выбор средств программной реализации разработанного метода оценки кредитоспособности физических лиц, описывается структура программного обеспечения, реализующего алгоритм градиентного бустинга деревьев решений, а также приводится описание формата входных и выходных данных и принципов взаимодействия компонентов системы.

3.1 Обоснование выбора технологий

Для реализации разработанного метода оценки кредитоспособности физических лиц были выбраны два основных инструмента: язык программирования Python и язык C#.

Python используется для реализации алгоритмов машинного обучения, обработки данных и построения модели кредитного скоринга, тогда как язык C# применяется для разработки пользовательского интерфейса, обеспечивающего взаимодействие пользователя с системой.

Язык Python выбран благодаря следующим преимуществам:

- высокая производительность при работе с числовыми данными и большими объемами информации;
- широкое распространение в задачах анализа данных, финансового моделирования и кредитного скоринга;
- наличие инструментов для визуализации и интерпретации результатов работы моделей;
- поддержка современных алгоритмов машинного обучения, включая градиентный бустинг.

Для разработки пользовательского интерфейса был выбран язык C# с использованием технологии Windows Forms. Данный выбор обусловлен следующими причинами:

- возможность быстрого создания настольных приложений с графическим интерфейсом;
- интеграция с операционной системой Windows;
- возможность взаимодействия с внешними процессами, включая запуск Python-скриптов.

В рамках реализации программного комплекса были использованы следующие основные библиотеки и инструменты:

- **pandas** – для обработки и анализа табличных данных;
- **numpy** – для выполнения численных операций;
- **scikit-learn** – для подготовки данных, оценки качества модели и расчета метрик;
- **LightGBM** – для реализации алгоритма градиентного бустинга деревьев решений;
- **matplotlib** – для визуализации результатов, включая ROC- и PR-кривые;
- **joblib** – для сохранения и загрузки обученной модели;
- **C# (Windows Forms)** – для разработки пользовательского интерфейса системы.

Выбор библиотеки LightGBM обусловлен ее высокой скоростью обучения, эффективной работой с большими объемами данных и встроенной поддержкой обработки пропущенных значений. Кроме того, данный алгоритм хорошо подходит для решения задач кредитного скоринга, поскольку способен выявлять сложные нелинейные зависимости между признаками и обеспечивает высокое качество классификации.

3.2 Основные программные модули системы

Разработанное программное обеспечение включает несколько взаимосвязанных модулей, реализующих подготовку данных, обучение модели машинного обучения и оценку кредитоспособности заемщика.

Структура программного комплекса и взаимодействие его основных компонентов представлены на рисунке 3.1.

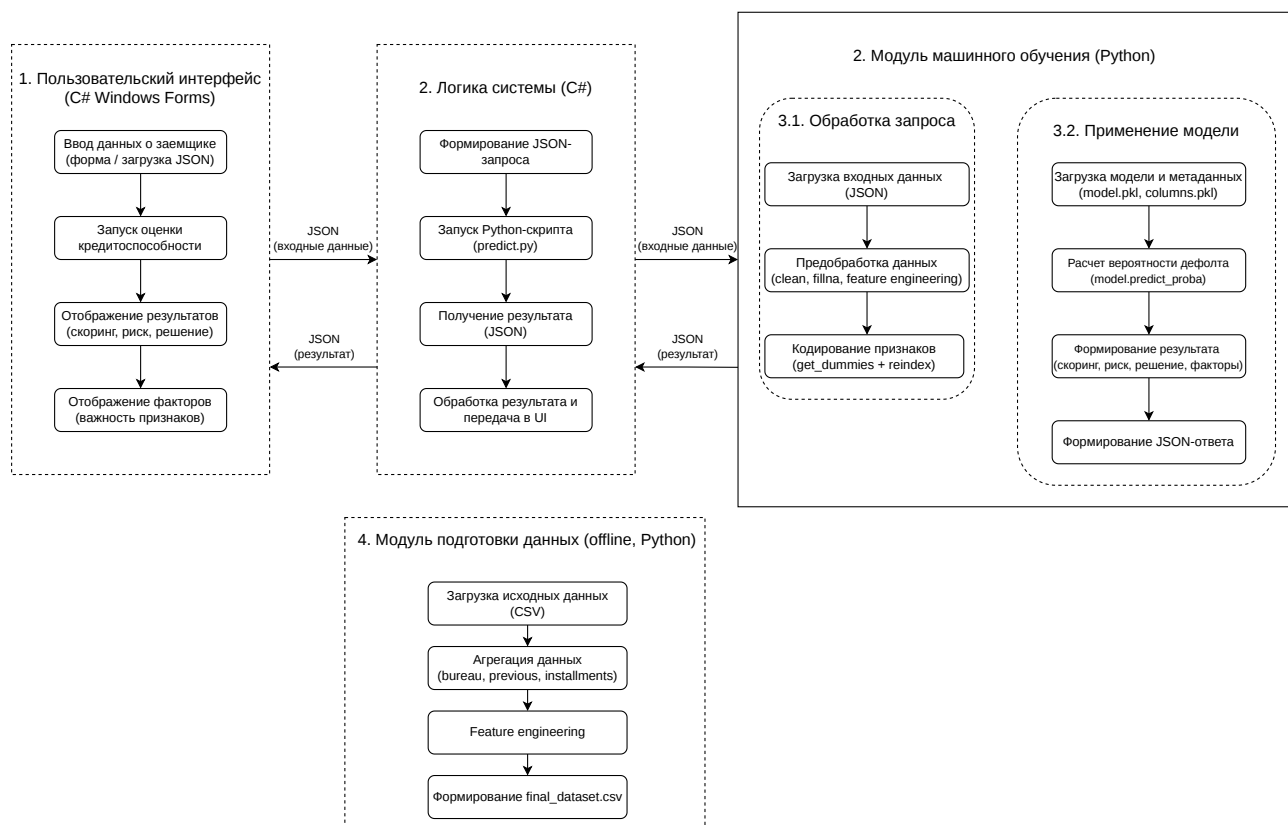


Рисунок 3.1 – Структурная схема взаимодействия компонентов системы оценки кредитоспособности

На рисунке представлена структура разработанной системы, а также схема взаимодействия между ее основными программными модулями. Система включает этапы подготовки данных, обучения модели, формирования прогнозов и отображения результатов пользователю.

Первым этапом работы системы является формирование набора данных для обучения модели. Для этого используется модуль предварительной обработки данных, реализованный на языке Python. Модуль выполняет загрузку исходных CSV-файлов, содержащих информацию о заемщиках, кредитной истории, предыдущих заявках и платежах по кредитам.

После загрузки выполняется агрегация данных по каждому заемщику, обработка пропущенных значений и формирование дополнительных признаков. В процессе feature engineering рассчитываются производные показатели, характеризующие долговую нагрузку клиента, уровень просрочек, соотношение кредита к доходу, кредитную активность и другие параметры. Итоговый набор дан-

ных сохраняется в файл `final_dataset.csv` и используется для последующего обучения модели.

Обучение модели реализовано в отдельном модуле на языке Python с использованием библиотеки LightGBM. Модуль выполняет загрузку подготовленного набора данных, разделение выборки на обучающую и тестовую части, кодирование категориальных признаков методом one-hot encoding и обучение модели градиентного бустинга деревьев решений.

В процессе обучения модели учитывалась проблема дисбаланса классов, характерная для задач кредитного скоринга, при которой количество заемщиков с дефолтом существенно меньше количества надежных клиентов. Для компенсации данного дисбаланса использовался параметр `scale_pos_weight`, позволяющий повысить чувствительность модели к выявлению рискованных заемщиков. Подбор оптимальных гиперпараметров осуществлялся с применением методов RandomizedSearchCV и GridSearchCV.

После завершения обучения выполняется расчет основных метрик качества модели, включая ROC-AUC, Precision, Recall и F1-score. Обученная модель сохраняется в файл `model.pkl`, а список признаков — в файл `columns.pkl`.

Для оценки кредитоспособности заемщика используется отдельный модуль предсказания. Данный модуль загружает обученную модель, принимает входные данные в формате JSON, выполняет предобработку и расчет вероятности дефолта с использованием функции `predict_proba`.

На основе полученной вероятности формируется скоринговый балл, уровень риска заемщика и рекомендации по принятию кредитного решения. Дополнительно реализован механизм интерпретации результатов с использованием библиотеки SHAP, позволяющий определить признаки, оказавшие наибольшее влияние на итоговое решение модели.

Пользовательский интерфейс системы реализован на платформе Windows Forms с использованием языка C#. Интерфейс обеспечивает ввод параметров заемщика, загрузку JSON-файлов и отображение результатов скоринга. Взаимодействие между C#-приложением и Python-модулем реализовано посредством запуска Python-скрипта через класс `ProcessStartInfo` с последующим чтением результата из стандартного потока вывода.

Полные листинги основных программных модулей приведены в приложении А.

3.3 Формат входных и выходных данных

Входные данные системы представляют собой информацию о заемщике, представленную в формате JSON.

Структура входных данных включает:

- финансовые характеристики (доход, сумма кредита, размер платежей);
- социально-демографические данные;
- параметры кредитной истории;
- агрегированные показатели поведения заемщика.

Выходные данные системы также формируются в формате JSON и содержат:

- вероятность дефолта;
- скоринговый балл;
- уровень кредитного риска;
- решение о выдаче кредита;
- факторы, влияющие на решение модели.

3.4 Тестирование программного обеспечения

Тестирование программы выполнено на реальных данных открытого набора Home Credit Default Risk, размещенного в открытом доступе на платформе Kaggle, а также на синтетически сформированных наборах данных, предназначенных для проверки различных сценариев функционирования программного комплекса [26].

Целью тестирования являлась проверка корректности работы программного комплекса и итоговой модели оценки кредитоспособности. Проверялись ввод данных заемщика, формирование входного JSON-файла, запуск Python-модуля предсказания, получение результата скоринга и отображение результата в пользовательском интерфейсе.

В ходе тестирования проверялись следующие сценарии:

- ручной ввод данных заемщика через формы приложения;
- формирование JSON-структуры на основе введенных данных;
- загрузка данных заемщика из JSON-файла;
- получение результата работы модели в формате JSON;
- отображение вероятности дефолта, скорингового балла и уровня риска;
- отображение факторов риска и рекомендаций;
- обработка ошибок при отсутствии файла или некорректных входных данных.

Для проверки качества итоговой модели исходный набор данных был разделен на обучающую и тестовую выборки в соотношении 80% и 20%. Тестовая выборка не использовалась при обучении модели, что позволило оценить качество работы метода на новых данных.

По результатам тестирования обученной модели были получены значения основных метрик качества, представленные в таблице 3.

Таблица 3 – Метрики качества итоговой модели

Метрика	Значение
Accuracy	0.48
ROC-AUC	0.777
Precision	0.12
Recall	0.89
F1-score	0.22

Полученные значения показывают, что модель обладает высокой полнотой выявления дефолтных заемщиков. Значение Recall составило 0.89, что означает способность модели обнаруживать значительную часть заемщиков с высоким риском дефолта. При этом значение Precision составило 0.12, что указывает на наличие большого количества ложноположительных решений.

Такая особенность является допустимой для рассматриваемой задачи, поскольку в кредитном скоринге пропуск заемщика с высоким риском дефолта может привести к финансовым потерям. Поэтому в рамках работы приоритет был отдан повышению полноты выявления дефолтных заемщиков.

Дополнительно была построена матрица ошибок, представленная в таблице 4.

Таблица 4 – Матрица ошибок итоговой модели

	Предсказано 0	Предсказано 1
Фактически 0	24972	31566
Фактически 1	561	4404

Из матрицы ошибок видно, что модель правильно выявила 4404 заемщика с дефолтом. Количество пропущенных дефолтных заемщиков составило 561. При этом 31566 надежных заемщиков были ошибочно отнесены к группе риска. Такой результат объясняется выбранным порогом классификации, равным 0.3, который был установлен для повышения полноты выявления дефолтов.

Для оценки влияния порога классификации был проведен дополнительный анализ количества ошибок при различных значениях порога. Результаты представлены в таблице 5.

Таблица 5 – Зависимость ошибок от порога классификации

Порог	FN	FP
0.30	561	31566
0.50	1563	15323
0.70	3112	4462

Анализ показал, что увеличение порога классификации снижает количество ложноположительных решений, однако одновременно увеличивает число пропущенных дефолтных заемщиков. Поэтому для итоговой модели был выбран порог 0.3. Такой порог позволяет уменьшить количество пропущенных дефолтов, что соответствует специфике задачи кредитного скоринга.

На рисунке 3.2 представлена ROC-кривая итоговой модели.

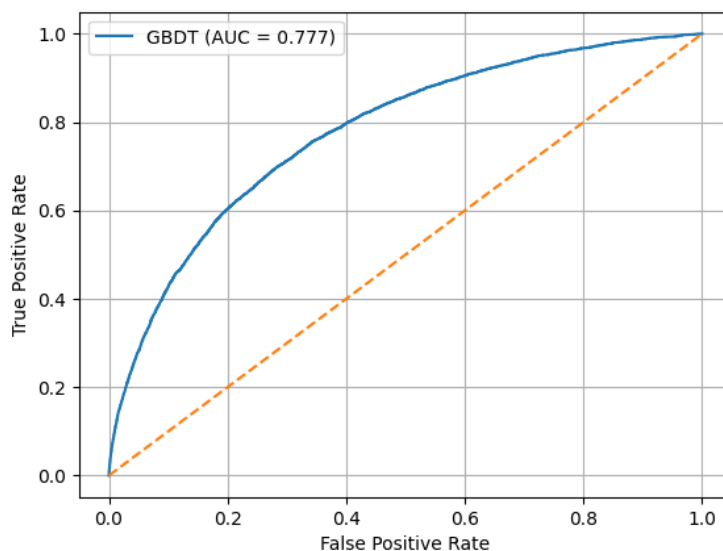


Рисунок 3.2 – ROC-кривая итоговой модели

ROC-кривая показывает, что модель обладает способностью разделять заемщиков по уровню кредитного риска. Значение ROC-AUC, равное 0.777, подтверждает, что модель выполняет ранжирование заемщиков лучше случайного классификатора.

На рисунке 3.3 представлена Precision-Recall кривая итоговой модели.

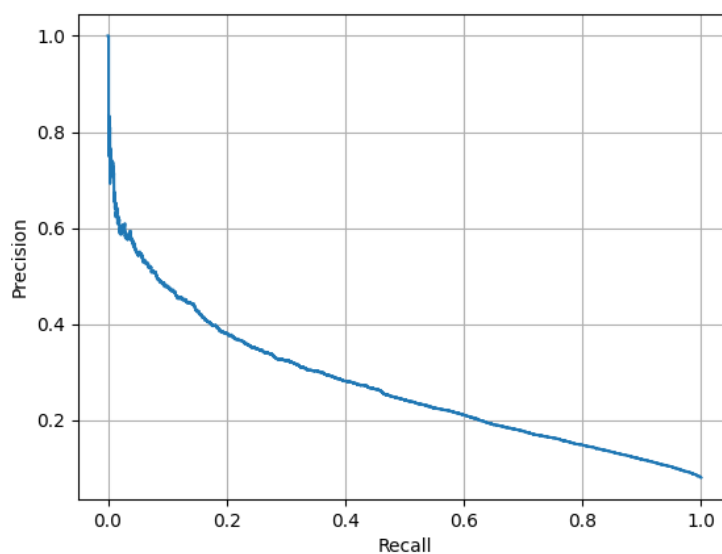


Рисунок 3.3 – Precision-Recall кривая итоговой модели

Precision-Recall кривая использовалась для оценки качества выявления по-

ложительного класса при наличии дисбаланса данных. Она показывает соотношение точности и полноты при изменении порога классификации.

Для проверки работы ручного ввода были подготовлены синтетические тестовые наборы данных, соответствующие различным профилям заемщиков: заемщик с низким уровнем риска, заемщик со средней кредитной нагрузкой, заемщик с высоким уровнем риска, пенсионер, а также заемщик с большим количеством просрочек и отказов. Использование таких наборов позволило проверить корректность заполнения всех полей формы и передачу данных в модель.

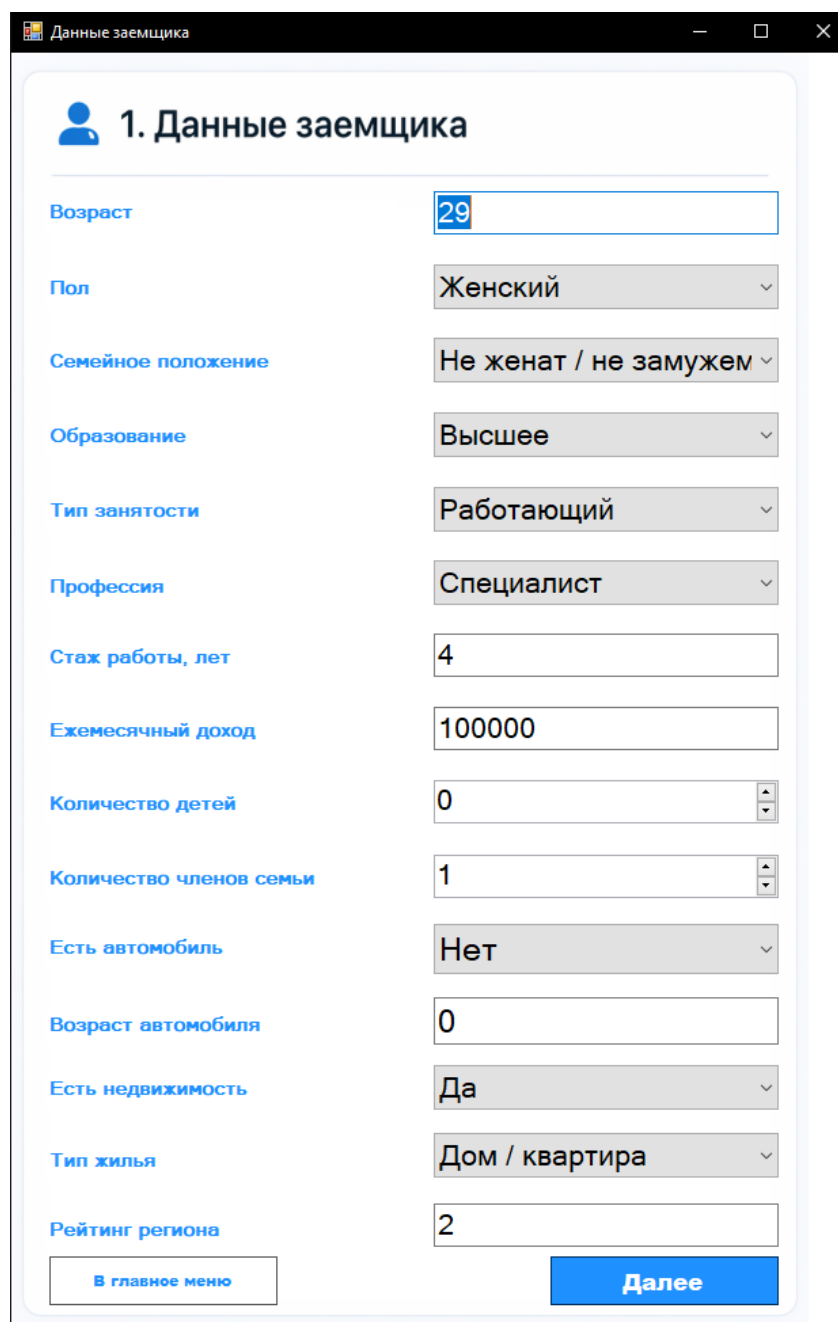
Результаты проведенного функционального тестирования разработанного программного обеспечения приведены в таблице 6.

Таблица 6 – Результаты функционального тестирования

Проверяемый сценарий	Ожидаемый результат	Результат
Ручной ввод данных заемщика	Данные принимаются системой и проходят проверку заполнения	Успешно
Формирование JSON по данным формы	Создается JSON-структура, соответствующая формату входных данных модели	Успешно
Загрузка данных из JSON-файла	Файл считывается, данные передаются в модуль предсказания	Успешно
Запуск Python-скрипта из C#	Python-модуль запускается из приложения и получает путь к входному файлу	Успешно
Получение результата скоринга	Приложение получает JSON с вероятностью дефолта, баллом и уровнем риска	Успешно
Отображение результата	На форме отображаются вероятность дефолта, скоринговый балл, уровень риска, факторы и рекомендации	Успешно
Обработка некорректных данных	Пользователю выводится сообщение об ошибке без аварийного завершения программы	Успешно

Примеры работы пользовательского интерфейса при выполнении тестирования представлены на рисунках 3.4–3.7.

На рисунке 3.4 показана форма ручного ввода данных заемщика.



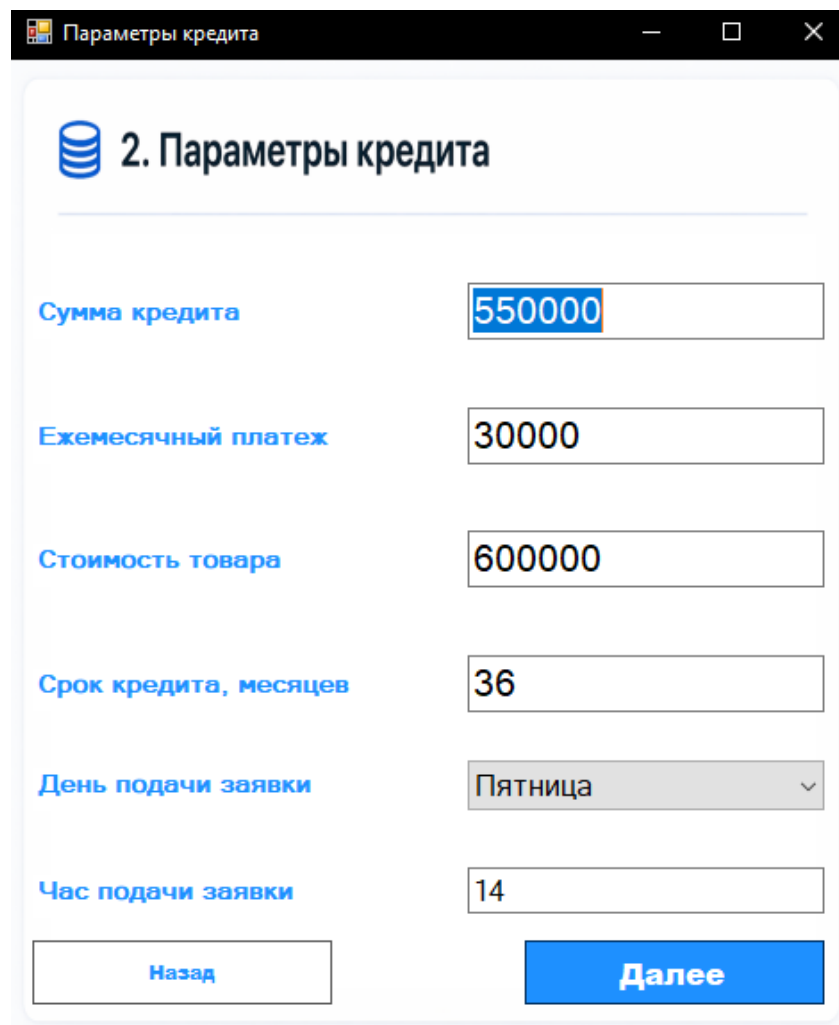
1. Данные заемщика

Возраст	29
Пол	Женский
Семейное положение	Не женат / не замужем
Образование	Высшее
Тип занятости	Работающий
Профессия	Специалист
Стаж работы, лет	4
Ежемесячный доход	100000
Количество детей	0
Количество членов семьи	1
Есть автомобиль	Нет
Возраст автомобиля	0
Есть недвижимость	Да
Тип жилья	Дом / квартира
Рейтинг региона	2

[В главное меню](#) [Далее](#)

Рисунок 3.4 – Форма ручного ввода данных заемщика

На рисунке 3.5 представлена форма ввода параметров кредита.



2. Параметры кредита

Сумма кредита	550000
Ежемесячный платеж	30000
Стоимость товара	600000
Срок кредита, месяцев	36
День подачи заявки	Пятница
Час подачи заявки	14

Назад Далее

Рисунок 3.5 – Форма ввода параметров кредита

На рисунке 3.6 представлена форма ввода сведений о кредитной истории и платежном поведении заемщика.

Кредитная история

3. Кредитная история и поведение

Общее количество кредитов	5
Количество активных кредитов	2
Количество закрытых кредитов	3
Были ли просрочки	Нет
Максимальная просрочка, дней	0
Доля просроченных платежей, %	5 %
Количество предыдущих заявок	4
Количество одобренных заявок	3
Количество отказанных заявок	1
Средняя сумма предыдущей заявки	300000
Средняя сумма предыдущего кредита	280000
Средний платеж по предыдущим заявкам	18000
Количество платежей	30
Количество просроченных платежей	2
Средний фактический платеж	18000
Средний плановый платеж	18000

Рисунок 3.6 – Форма ввода сведений о кредитной истории заемщика

На рисунке 3.7 показан результат скоринга, полученный после обработки введенных данных, а также JSON-структура, сформированная программой при ручном вводе и переданная в модуль предсказания.

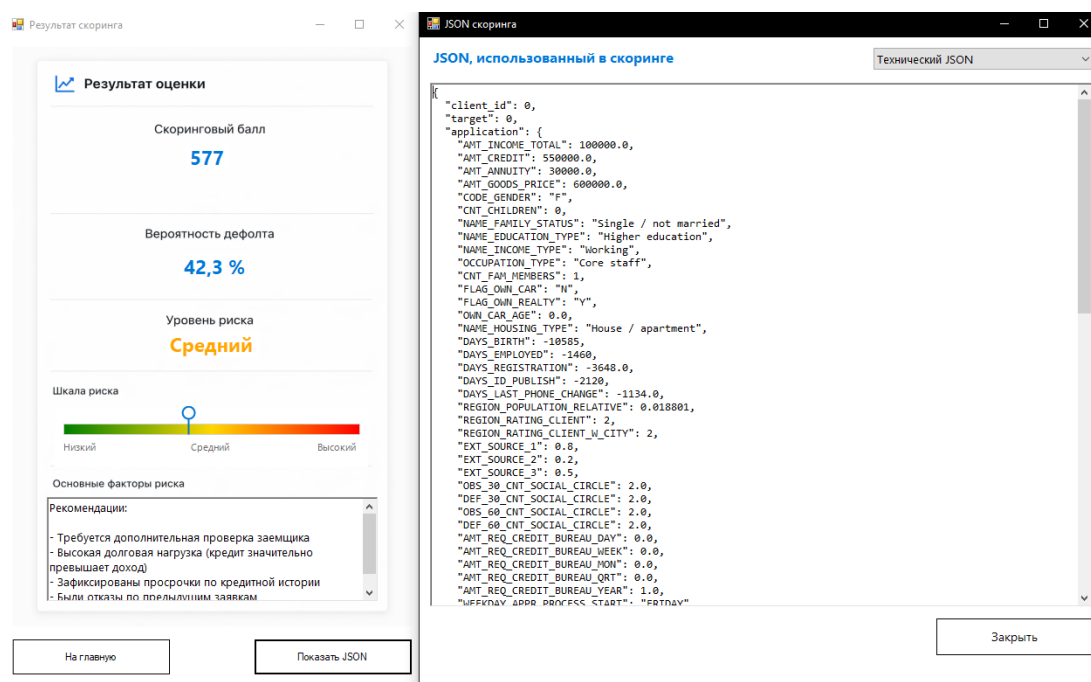


Рисунок 3.7 – Форма отображения результата скоринга и просмотр сформированного JSON

Таким образом, тестирование подтвердило корректность работы основных компонентов программного комплекса. Итоговая модель корректно рассчитывает вероятность дефолта, а приложение обеспечивает ввод данных, формирование JSON, запуск Python-модуля, получение результата и отображение скоринговой оценки пользователю.

Выводы

В данном разделе были рассмотрены вопросы программной реализации разработанного метода оценки кредитоспособности физических лиц.

Обоснован выбор языка программирования Python для реализации модулей обработки данных, обучения и предсказания, а также языка C# и технологии Windows Forms для разработки пользовательского интерфейса.

Описана структура программного комплекса, включающая модуль формирования набора данных, модуль обучения модели, модуль предсказания кредитоспособности и пользовательский интерфейс.

Определены форматы входных и выходных данных, используемые при взаимодействии между приложением и Python-модулем предсказания.

Проведено тестирование программного обеспечения на тестовой выборке и синтетических наборах данных. Результаты тестирования подтвердили корректность формирования входного JSON, запуска Python-модуля, получения результата скоринга и отображения итоговой оценки пользователю.

4 Исследовательский раздел

В данном разделе проводится исследование эффективности разработанного метода оценки кредитоспособности физических лиц на основе алгоритма градиентного бустинга деревьев решений (GBDT).

Цель исследования заключается в оценке качества разработанной модели и сравнении ее результатов с традиционными алгоритмами машинного обучения, применяемыми в задачах кредитного скоринга.

Для достижения поставленной цели решались следующие задачи:

- анализ дисбаланса классов в исходном наборе данных;
- сравнение качества моделей логистической регрессии, случайного леса и градиентного бустинга;
- оценка качества моделей с использованием метрик ROC-AUC, Precision, Recall и F1-score;
- анализ влияния порога классификации на качество выявления дефолтов;
- анализ наиболее значимых признаков модели.

Home Credit Default Risk представляет собой открытый набор данных, размещенный на платформе Kaggle, и использовался в работе в качестве исходных данных. Набор данных содержит сведения о заявках клиентов, кредитной истории, предыдущих заявках и платежной дисциплине заемщиков. Положительным классом в рассматриваемой задаче является наступление дефолта заемщика, то есть значение целевой переменной $TARGET = 1$ [26].

Перед обучением моделей категориальные признаки были преобразованы методом one-hot encoding. Исходный набор данных был разделен на обучающую и тестовую выборки в соотношении 80% и 20%. Разделение выполнялось с сохранением соотношения классов с использованием стратификации. Оценка качества моделей проводилась на тестовой выборке, не использовавшейся при обучении.

Для учета дисбаланса классов при обучении модели LightGBM использовался параметр `scale_pos_weight`, рассчитываемый как отношение количества

объектов отрицательного класса к количеству объектов положительного класса в обучающей выборке.

В качестве сравниваемых моделей были выбраны:

1. Логистическая регрессия;
2. Случайный лес (Random Forest);
3. Градиентный бустинг (LightGBM).

Выбранные модели позволяют сравнить линейный подход, ансамблевый метод на основе независимых деревьев и метод градиентного бустинга.

4.1 Анализ дисбаланса классов

Для исходного набора данных характерен существенный дисбаланс классов: доля клиентов с дефолтом значительно ниже доли надежных заемщиков. Доля положительного класса составляет около 8%, что означает, что модель, всегда предсказывающая отсутствие дефолта, будет иметь высокое значение Accurasy, но не будет решать основную задачу выявления проблемных заемщиков.

В связи с этим метрика Accurasy не может рассматриваться как основная метрика качества, поскольку при выраженном дисбалансе классов она может давать завышенную оценку качества модели за счет преобладания объектов большинства класса. Для более корректной оценки моделей в таких условиях использовались метрики, учитывающие качество выявления положительного класса [27]:

- ROC-AUC;
- Precision;
- Recall;
- F1-score;
- Precision-Recall кривая.

Особое внимание уделялось метрике Recall для положительного класса, поскольку в задаче кредитного скоринга пропуск заемщика с высоким риском дефолта может привести к финансовым потерям банка.

4.2 Оптимизация гиперпараметров модели

Для повышения качества разработанной модели была проведена оптимизация гиперпараметров алгоритма градиентного бустинга LightGBM. В рамках исследования рассматривались два подхода к подбору параметров: случайный поиск RandomizedSearchCV и полный перебор GridSearchCV.

RandomizedSearchCV выполняет проверку заданного количества случайно выбранных комбинаций гиперпараметров из общего пространства поиска. Такой подход позволяет сократить вычислительные затраты по сравнению с полным перебором, поскольку число проверяемых комбинаций задается заранее. Метод GridSearchCV, в свою очередь, выполняет исчерпывающий перебор всех комбинаций параметров из заданной сетки, что обеспечивает полный поиск по выбранному пространству, но требует значительно больше времени при увеличении числа параметров и их возможных значений.

В процессе оптимизации подбирались следующие параметры:

- количество деревьев (n_estimators);
- скорость обучения (learning_rate);
- количество листьев (num_leaves);
- максимальная глубина деревьев (max_depth);
- минимальное количество объектов в листе (min_child_samples).

Подбор параметров осуществлялся с использованием трехкратной кросс-валидации с оценкой качества по метрике ROC-AUC, поскольку данная метрика оценивает способность модели ранжировать объекты разных классов и менее чувствительна к выбору конкретного порога классификации, чем Accuracy.

Для RandomizedSearchCV использовалось расширенное пространство гиперпараметров. Метод случайного поиска проверял ограниченное количество

случайно выбранных комбинаций, что позволило выполнить подбор параметров за приемлемое время и избежать полного перебора всех возможных сочетаний.

В результате применения `RandomizedSearchCV` было получено значение F1-score, равное 0.2173 и ROC-AUC, равное 0.7758. Полученные результаты показывают, что случайный поиск позволил улучшить значение F1-score по сравнению с базовой конфигурацией модели.

Дополнительно был выполнен подбор гиперпараметров с использованием `GridSearchCV` на сокращенном пространстве параметров. Такой подход позволил выполнить полный перебор в рамках ограниченной сетки и сравнить его с результатами случайного поиска без чрезмерного увеличения времени вычислений.

В результате применения `GridSearchCV` на сокращенной сетке были получены следующие параметры:

- `learning_rate = 0.03`;
- `max_depth = 8`;
- `min_child_samples = 100`;
- `n_estimators = 500`;
- `num_leaves = 31`.

Значение метрики ROC-AUC для `GridSearchCV` составило 0.7771, значение F1-score составило 0.2152. Таким образом, полный перебор на сокращенном пространстве параметров позволил получить наибольшее значение ROC-AUC среди рассмотренных вариантов, однако прирост относительно базовой модели оказался незначительным.

Для сравнения также была оценена базовая конфигурация модели без автоматического подбора гиперпараметров. Значение ROC-AUC для базовой модели составило 0.7757, значение F1-score составило 0.2098.

Дополнительно оценена вычислительная сложность полного перебора гиперпараметров на расширенном пространстве параметров. При использовании 108 комбинаций параметров и трехкратной кросс-валидации общее количество

запусков обучения модели составило бы 324. С учетом экспериментально измеренного времени обучения одной комбинации параметров полный перебор потребовал бы значительно большего времени, чем случайный поиск.

GridSearchCV при расширенном пространстве параметров не является целесообразным решением в условиях ограниченных вычислительных ресурсов. RandomizedSearchCV позволяет получить сопоставимое качество модели при меньшем количестве запусков обучения, что делает данный метод практически полезным при подборе гиперпараметров.

Результаты сравнения различных методов оптимизации гиперпараметров представлены на рисунке 4.1.

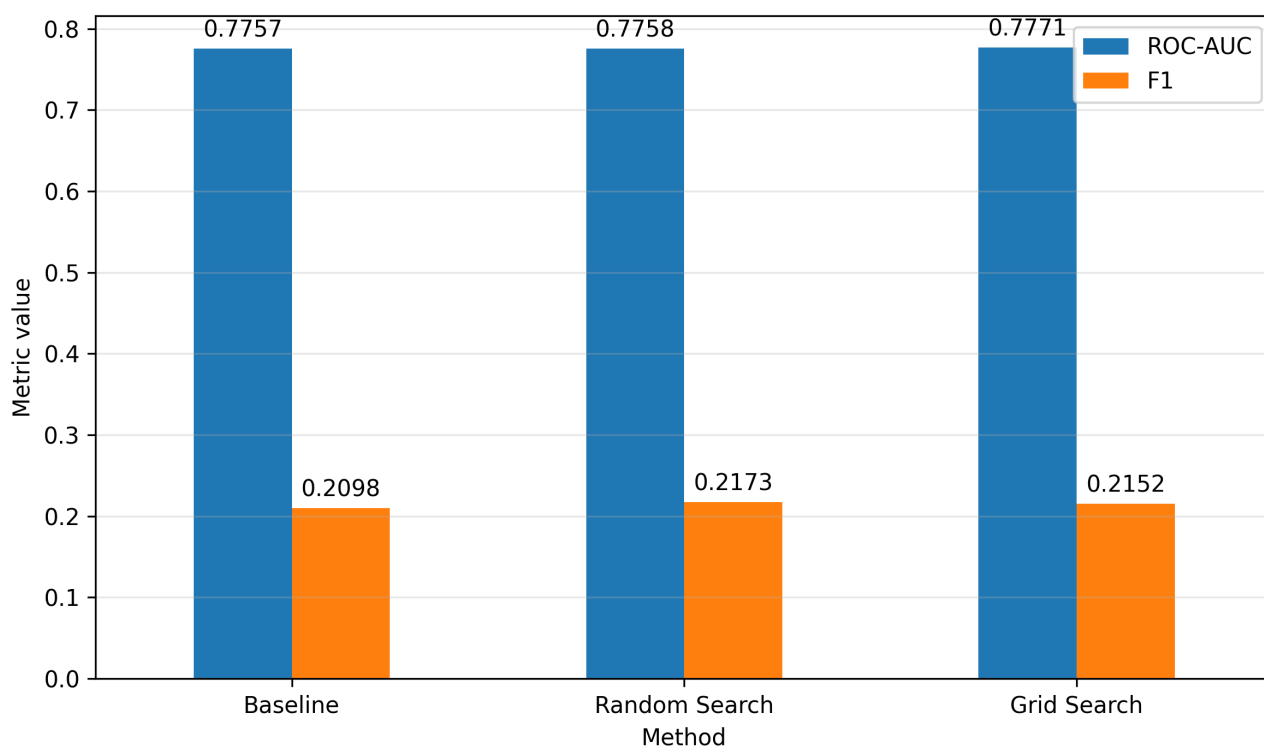


Рисунок 4.1 – Сравнение методов оптимизации гиперпараметров

По результатам, представленным на графике, значения основной метрики у всех рассмотренных методов оптимизации находятся на близком уровне. Базовая модель без оптимизации показывает значение 0.7757, GridSearchCV — 0.7771, а RandomizedSearchCV — 0.7758. Таким образом, полный перебор параметров позволил получить наибольшее значение ROC-AUC, однако улучшение относительно базовой модели оказалось незначительным.

Для метрики F1-score наилучший результат был получен при использовании RandomizedSearchCV и составил 0.2173. Базовая модель показала значение

0.2098, а GridSearchCV — 0.2152. Это показывает, что случайный поиск позволил улучшить баланс между Precision и Recall по сравнению с базовой конфигурацией.

Таким образом, можно сделать вывод, что оба метода оптимизации гиперпараметров позволяют получить сопоставимое качество модели. GridSearchCV обеспечивает немного большее значение ROC-AUC, RandomizedSearchCV показывает лучшее значение F1-score и требует меньшего количества проверяемых комбинаций. Поэтому при ограниченных вычислительных ресурсах случайный поиск является более предпочтительным, тогда как полный перебор целесообразно использовать только на сокращенном пространстве параметров.

4.3 Сравнение моделей по основным метрикам

Для сравнения моделей использовались метрики ROC-AUC и F1-score. ROC-AUC показывает, насколько хорошо модель различает заемщиков с разным уровнем кредитного риска независимо от выбранного порога классификации. Метрика F1-score оценивает баланс между Precision и Recall для положительного класса, которым в данной задаче считается дефолт заемщика.

Результаты сравнения моделей по основным метрикам представлены на рисунке 4.2.

По метрике ROC-AUC наилучший результат показала модель LightGBM. Значение ROC-AUC для нее составило 0.78. Логистическая регрессия показала значение 0.75, а случайный лес — 0.73. Это означает, что модель LightGBM лучше остальных рассмотренных алгоритмов ранжирует заемщиков по вероятности дефолта.

По F1-score метрике лучший результат был получен для LightGBM. Значение F1-score составило 0.22. Для логистической регрессии значение F1-score составило 0.18, для случайного леса — 0.14. Следовательно, при выбранном пороге классификации LightGBM обеспечивает более удачный баланс между выявлением дефолтных заемщиков и количеством ложноположительных срабатываний.

Таким образом, по совокупности ROC-AUC и F1-score модель LightGBM показала наиболее предпочтительный результат среди рассмотренных алгоритмов.

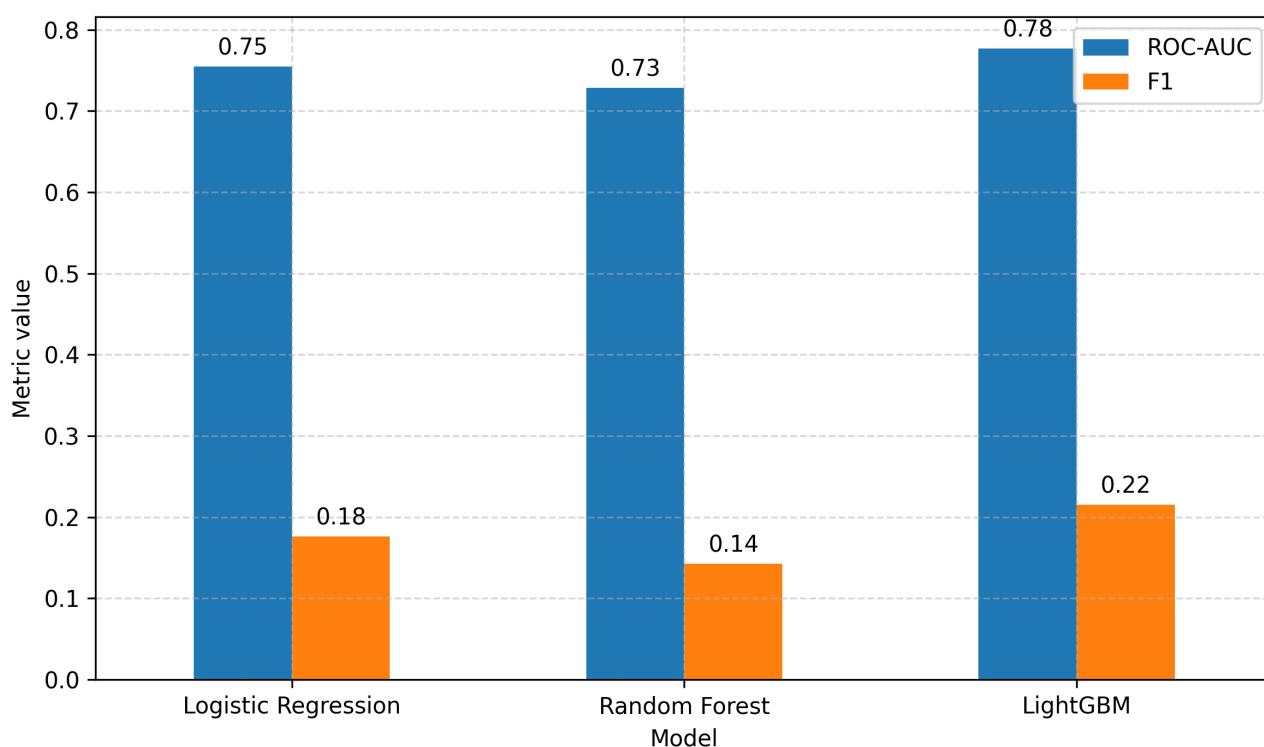


Рисунок 4.2 – Сравнение моделей по основным метрикам

4.4 Сравнение моделей по дополнительным метрикам

Дополнительно были рассмотрены метрики Accuracy, Precision и Recall. Их значения представлены на рисунке 4.3.

Высокие значения Accuracy были получены для логистической регрессии и случайного леса: 0.91 и 0.92 соответственно. Однако в данной задаче высокая Accuracy не является достаточным показателем качества модели. Это связано с дисбалансом классов: надежных заемщиков в выборке значительно больше, чем заемщиков с дефолтом. Поэтому модель может часто предсказывать отсутствие дефолта и получать высокий общий процент правильных ответов, но при этом плохо выявлять проблемных клиентов.

Значения Recall для логистической регрессии и случайного леса составили 0.12 и 0.09 соответственно. Это означает, что данные модели находят только небольшую часть заемщиков, фактически допустивших дефолт. Для задачи кредитного скоринга такой результат является недостаточным, поскольку пропуск рискованного заемщика может привести к финансовым потерям банка.

Модель LightGBM, напротив, показала существенно более высокое значение Recall, равное 0.89. Это означает, что модель выявляет большую часть за-

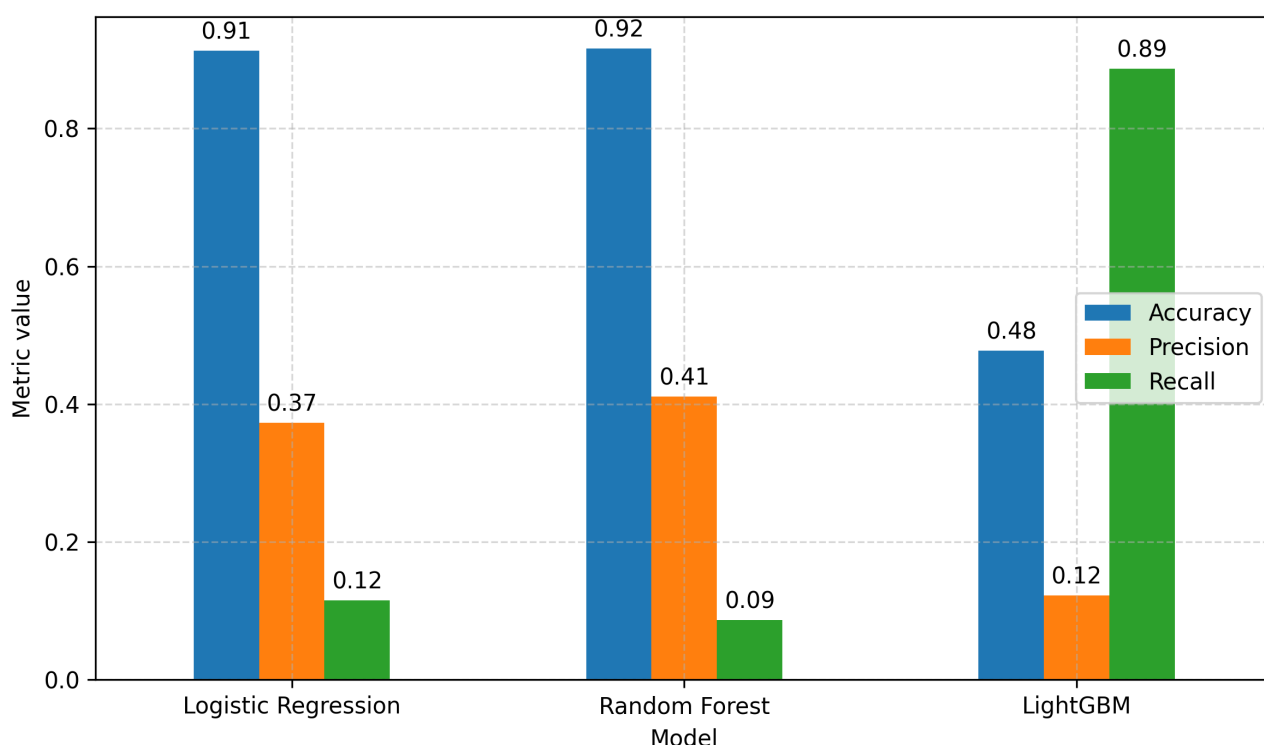


Рисунок 4.3 – Сравнение моделей по дополнительным метрикам

емщиков с высоким риском дефолта. При этом значение Accuracy у LightGBM составило 0.48, а Precision — 0.12. Низкое значение Precision связано с тем, что при выбранном пороге классификации модель чаще относит заемщиков к группе риска, увеличивая количество ложноположительных решений.

Для задачи кредитного скоринга такой результат является ожидаемым: чем больше дефолтных заемщиков выявляет модель, тем чаще она ошибочно относит к группе риска надежных клиентов. В данной работе использовался порог классификации 0.3, что позволило повысить Recall и уменьшить количество пропущенных дефолтных заемщиков. Такой выбор порога оправдан тем, что для банка пропуск проблемного заемщика является более критичным, чем направление надежного клиента на дополнительную проверку.

Следовательно, метрика Accuracy не может рассматриваться как основная метрика качества в данной задаче. Более важными являются ROC-AUC, Recall и F1-score, так как они позволяют оценить способность модели работать с редким положительным классом.

4.5 Анализ ROC-кривых

Были построены ROC-кривые сравниваемых моделей для дополнительной оценки качества классификации. Данный график позволяет оценить поведение модели не при одном фиксированном пороге классификации, а при изменении порога в диапазоне от 0 до 1.

ROC-кривая показывает зависимость доли верно выявленных положительных объектов от доли ложноположительных срабатываний при изменении порога классификации. В рассматриваемой задаче положительным классом является дефолт заемщика. Диагональная линия на графике соответствует случайному классификатору, а расположение кривой выше данной диагонали указывает на более высокую способность модели различать классы.

ROC-кривые сравниваемых моделей представлены на рисунке 4.4.

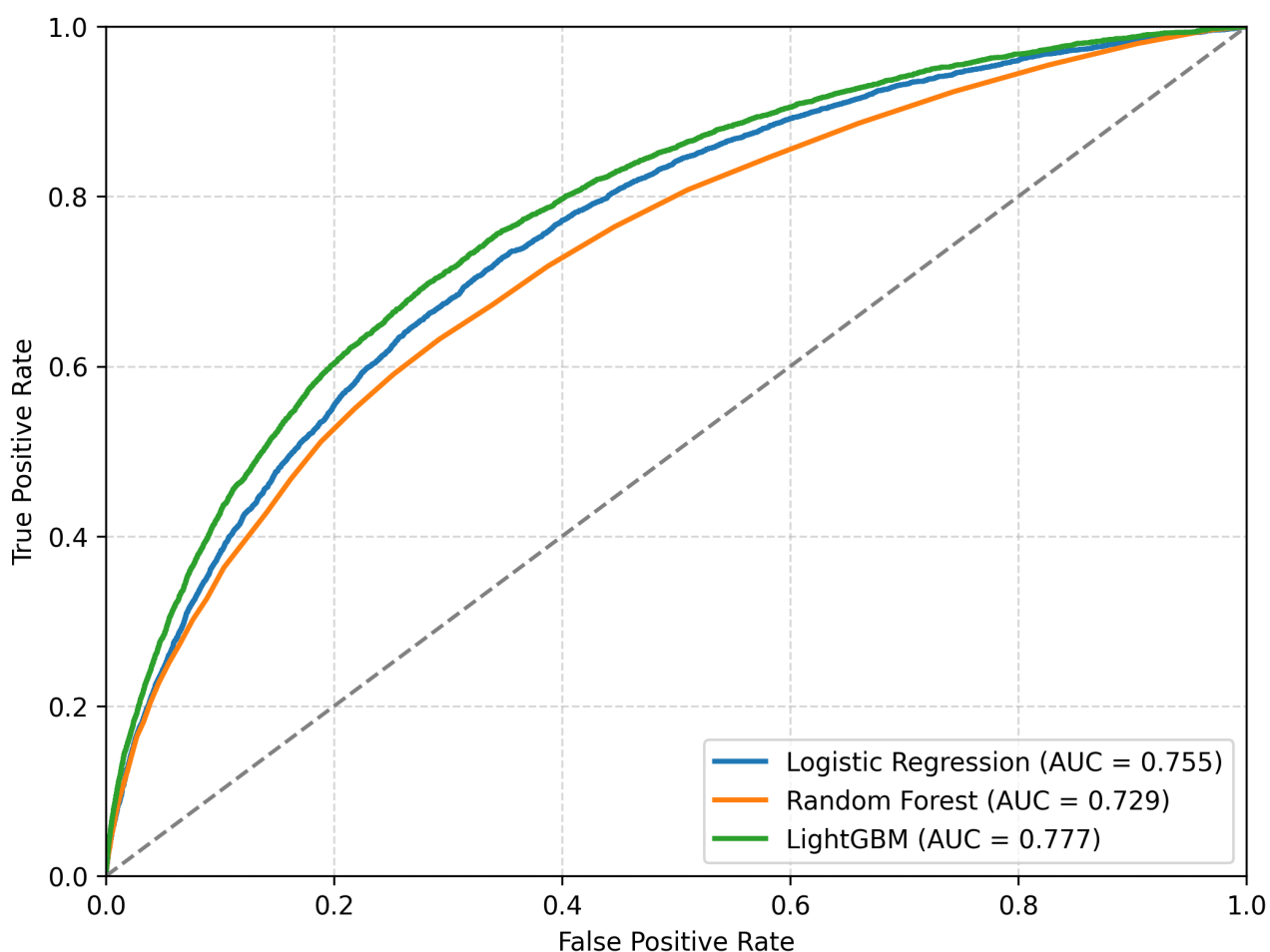


Рисунок 4.4 – ROC-кривые моделей

Из рисунка видно, что ROC-кривая модели LightGBM расположена вы-

ше кривых логистической регрессии и случайного леса на большей части диапазона значений False Positive Rate. Это соответствует наибольшему значению ROC-AUC, равному 0.777. Логистическая регрессия занимает промежуточное положение с ROC-AUC 0.755, а случайный лес показывает наименьшее значение ROC-AUC 0.729.

Полученные результаты подтверждают преимущество LightGBM. Данная модель лучше ранжирует заемщиков по уровню кредитного риска среди рассмотренных алгоритмов.

4.6 Анализ Precision-Recall кривых

Для детальной оценки качества моделей были построены Precision-Recall кривые. В отличие от ROC-кривой, данный график особенно полезен при несбалансированных классах, так как показывает качество модели именно при выявлении положительного класса.

В данной задаче положительный класс соответствует дефолту заемщика. Recall показывает долю выявленных дефолтных заемщиков, а Precision – долю действительно дефолтных заемщиков среди тех, кого модель отнесла к рискованным.

Precision-Recall кривые представлены на рисунке 4.5.

Базовый уровень на графике соответствует доле положительного класса в выборке и составляет около 0.08. Расположение кривых выше данного уровня означает, что модели дают результат лучше случайного выбора заемщиков.

Модель LightGBM показывает наиболее устойчивый баланс между Recall и Precision. При увеличении Recall значение Precision у данной модели остается выше, чем у логистической регрессии и случайного леса на значительной части графика. Это означает, что LightGBM лучше подходит для выявления заемщиков с повышенным риском дефолта при разных значениях порога классификации.

Логистическая регрессия показывает результат, близкий к LightGBM на отдельных участках, однако в целом уступает ей по качеству ранжирования. Случайный лес располагается ниже остальных моделей на большей части диапазона Recall, что согласуется с его более низкими значениями ROC-AUC и F1-score.

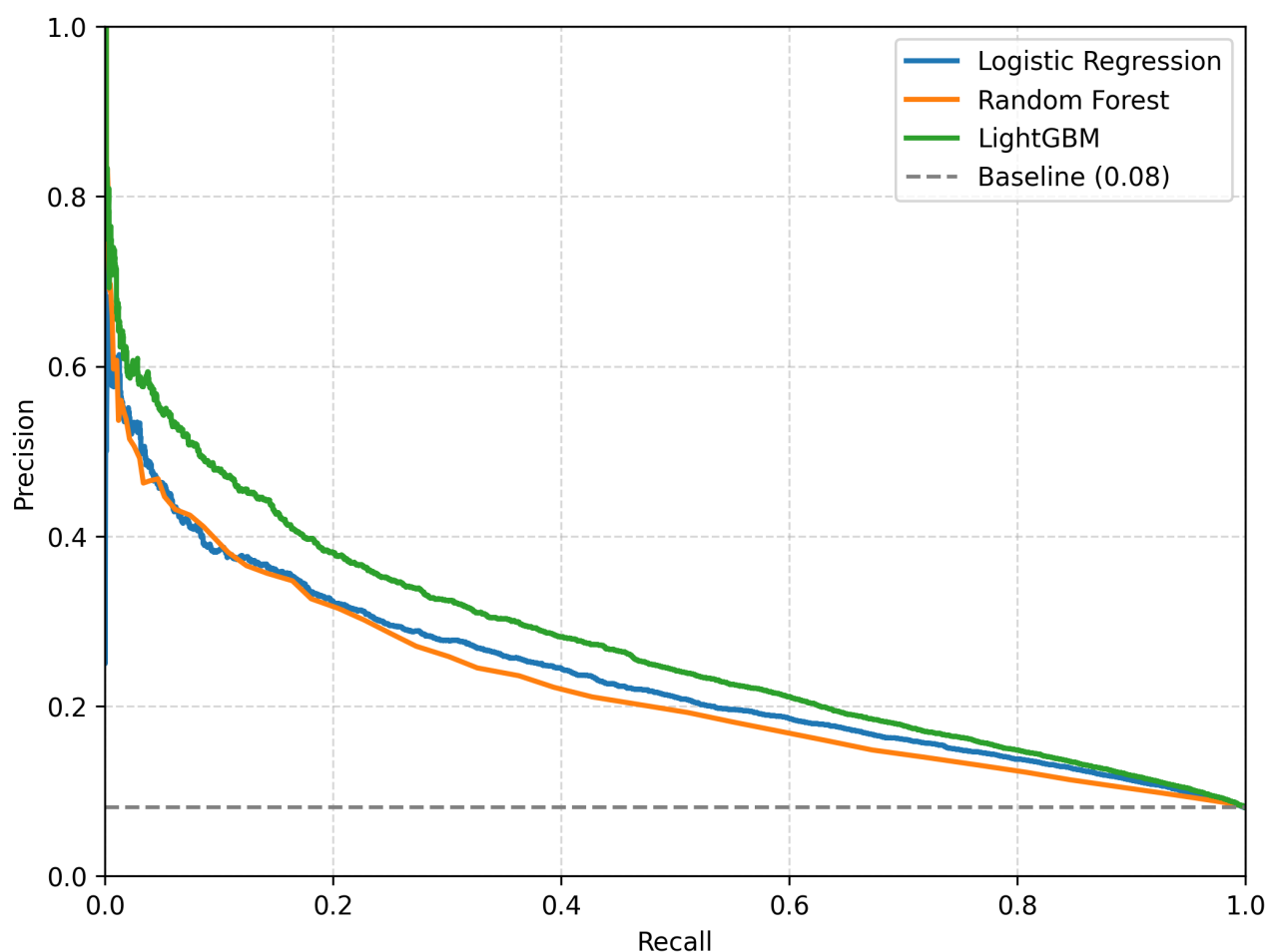


Рисунок 4.5 – Precision-Recall кривые моделей

4.7 Анализ значимости признаков

Для интерпретации результатов модели была проведена оценка значимости признаков LightGBM. Наиболее значимые признаки представлены на рисунке 4.6.

На результат модели больше всего повлияли внешние скоринговые признаки EXT_SOURCE_1, EXT_SOURCE_2 и EXT_SOURCE_3. Высокая значимость данных признаков объясняется тем, что они уже содержат агрегированную информацию о надежности заемщика, полученную из внешних источников.

Значимыми факторами также являются возраст заемщика, сумма кредита, ежемесячный платеж, стаж занятости и признаки предыдущих заявок. Эти признаки содержательно связаны с кредитным риском: возраст и занятость характеризуют социально-экономический профиль клиента, сумма кредита и платеж

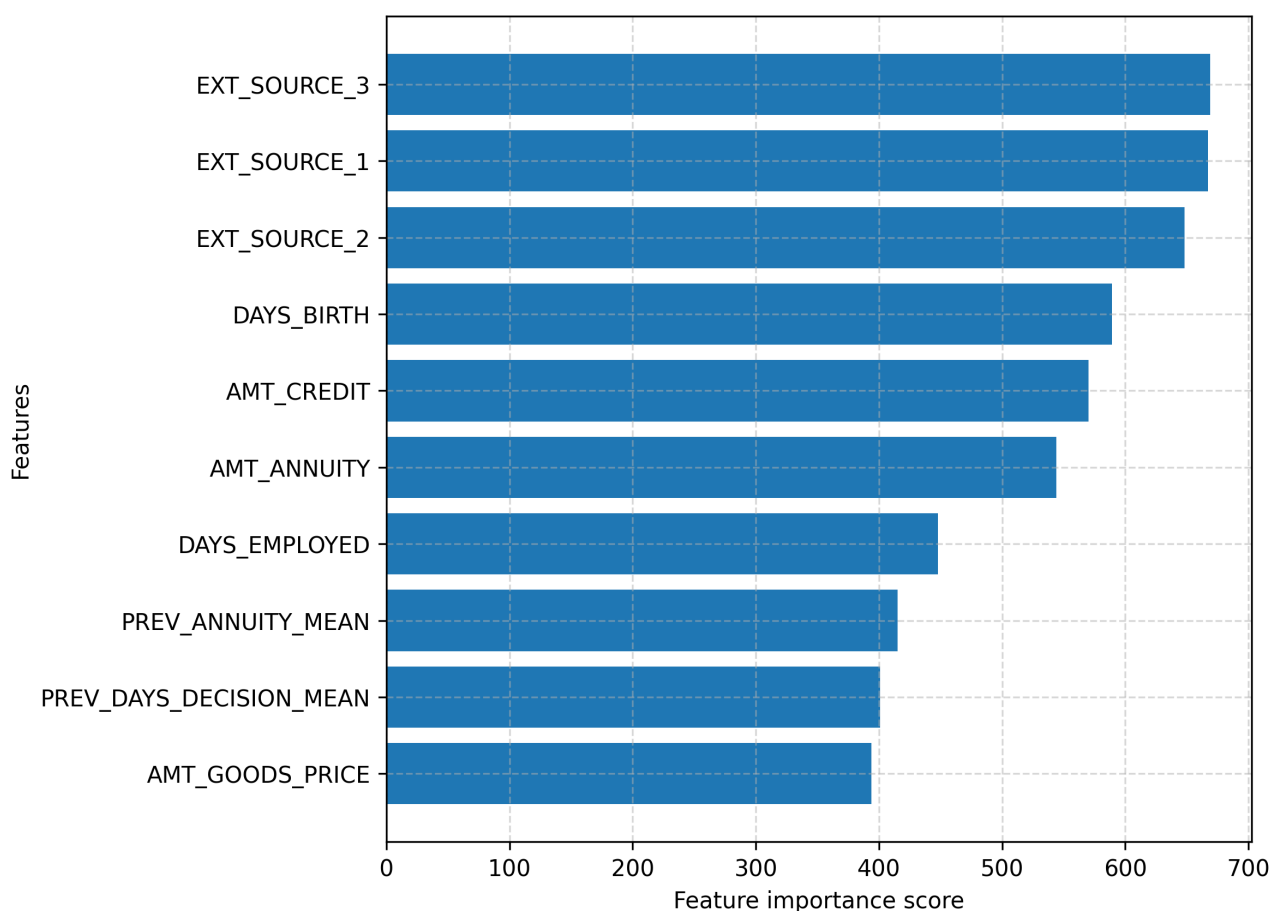


Рисунок 4.6 – Наиболее значимые признаки модели

отражают долговую нагрузку, а история предыдущих заявок показывает прошлое кредитное поведение заемщика.

Полученные результаты подтверждают, что модель использует признаки, имеющие содержательную связь с предметной областью кредитного скоринга.

4.8 Ограничения проведенного исследования

В рамках проведенного исследования модель обучалась на статическом наборе данных, в котором признаки рассматриваются как фиксированные характеристики заемщика и его кредитной истории. Однако в реальных банковских системах кредитоспособность клиента может изменяться во времени.

К временным факторам относятся изменение дохода клиента, динамика кредитной нагрузки, появление новых заявок, изменение платежной дисциплины и возникновение просрочек. Учет таких изменений позволяет отслеживать ухудшение финансового состояния заемщика и повышать качество прогнози-

вания дефолта.

В рамках данной работы временные зависимости учитывались косвенно через агрегированные признаки, например показатели по предыдущим заявкам, кредитной истории и платежам. Однако полноценное моделирование временной динамики не проводилось.

Таким образом, одним из ограничений проведенного исследования является отсутствие полноценного учета временной динамики. Учет временных факторов может рассматриваться как направление дальнейшего развития разработанного метода.

Выводы

В ходе исследования была проведена оценка качества разработанного метода оценки кредитоспособности физических лиц на основе алгоритма градиентного бустинга LightGBM. Для проверки эффективности метода были рассмотрены несколько моделей машинного обучения: логистическая регрессия, случайный лес и градиентный бустинг.

Результаты экспериментов показали, что модель LightGBM обеспечивает наилучшее качество среди рассмотренных алгоритмов по метрикам ROC-AUC и F1-score. Это свидетельствует о более высокой способности модели различать заемщиков с различным уровнем кредитного риска и более эффективно выявлять клиентов с вероятностью дефолта.

Было установлено, что в условиях дисбаланса классов метрика Accuracy не является достаточной для оценки качества модели. Логистическая регрессия и случайный лес показали высокие значения Accuracy, однако имели низкое значение Recall для положительного класса. Это означает, что данные модели пропускали значительную часть дефолтных заемщиков.

Использование пониженного порога классификации позволило повысить полноту выявления заемщиков с высоким риском дефолта. При этом снижение порога привело к уменьшению Precision, что связано с ростом числа ложноположительных решений. Такой результат отражает характерный для кредитного скоринга компромисс между снижением кредитного риска и сохранением числа одобренных заявок.

Анализ значимости признаков показал, что наибольшее влияние на ре-

зультат модели оказывают внешние скоринговые оценки, возраст заемщика, сумма кредита, размер ежемесячного платежа, стаж занятости и признаки, связанные с предыдущими заявками. Данные признаки имеют содержательную связь с предметной областью и отражают основные факторы, влияющие на кредитный риск.

Таким образом, проведенное исследование подтвердило целесообразность использования алгоритма градиентного бустинга LightGBM для задачи оценки кредитоспособности физических лиц. Разработанный метод может применяться как инструмент предварительной оценки кредитного риска, при этом итоговое решение о выдаче кредита должно приниматься с учетом дополнительной проверки и экспертной оценки специалиста.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была рассмотрена задача оценки кредитоспособности физических лиц в банковских системах с использованием алгоритма градиентного бустинга деревьев решений.

В аналитическом разделе был проведен анализ предметной области кредитного скоринга. Были рассмотрены основные понятия, связанные с оценкой кредитного риска, дефолтом заемщика, скоринговым баллом и процессом принятия кредитного решения. Также были проанализированы традиционные статистические методы и современные алгоритмы машинного обучения, применяемые в задачах кредитного скоринга. В результате анализа было обосновано использование алгоритма градиентного бустинга деревьев решений как метода, способного учитывать сложные нелинейные зависимости между признаками заемщика и вероятностью дефолта. Была сформулирована цель работы и выполнена формализация задачи оценки кредитоспособности с использованием IDEF0-диаграмм. В результате были выделены основные элементы процесса оценки кредитоспособности: исходная информация о заемщике, правила и ограничения, используемые средства обработки данных, а также итоговые результаты скоринговой оценки.

В технологическом разделе описан программный комплекс для оценки кредитоспособности физических лиц. В состав программного комплекса входят модуль формирования набора данных, модуль обучения модели, модуль предсказания кредитоспособности и пользовательский интерфейс. Для реализации модулей обработки данных и машинного обучения был использован язык Python, а для разработки графического интерфейса — язык C# и технология Windows Forms.

В рамках разработки были определены форматы входных и выходных данных. Входные данные представляются в формате JSON и содержат финансовые, социально-демографические характеристики заемщика, сведения о кредитной истории и агрегированные показатели платежного поведения. Выходные данные включают вероятность дефолта, скоринговый балл, уровень кредитного риска, решение модели, факторы риска и рекомендации. Проведено тестирование программного обеспечения на тестовой выборке и синтетических наборах данных. В ходе тестирования проверены основные функции программного ком-

плекса: ручной ввод данных, формирование JSON-структуры, запуск модуля предсказания из приложения, получение результата скоринга и вывод итоговой оценки в интерфейсе. Результаты тестирования подтвердили корректность работы основных компонентов программного комплекса.

В исследовательском разделе была проведена оценка эффективности разработанного метода. Для сравнения были рассмотрены логистическая регрессия, случайный лес и модель градиентного бустинга LightGBM. Экспериментальные результаты показали, что модель LightGBM обеспечивает наилучшее качество среди рассмотренных алгоритмов по метрикам ROC-AUC и F1-score. Было установлено, что в условиях дисбаланса классов метрика Ассигасу не является достаточной для оценки качества модели. Использование пониженного порога классификации позволило повысить полноту выявления заемщиков с высоким риском дефолта. При этом снижение порога привело к увеличению числа ложноположительных решений, что является характерным компромиссом для задач кредитного скоринга.

Был выполнен анализ значимости признаков модели. Наибольшее влияние на результат оказали внешние скоринговые оценки, возраст заемщика, сумма кредита, размер ежемесячного платежа, стаж занятости и признаки, связанные с предыдущими заявками. Полученные результаты соответствуют предметной области кредитного скоринга и подтверждают, что модель использует содержательно значимые факторы кредитного риска.

Таким образом, поставленная цель работы была достигнута: был разработан метод оценки кредитоспособности физических лиц на основе алгоритма градиентного бустинга деревьев решений и создан программный комплекс, реализующий данный метод. Разработанное программное средство может использоваться как инструмент предварительной оценки кредитного риска заемщика, при этом итоговое решение о выдаче кредита должно приниматься с учетом дополнительной проверки и экспертной оценки специалиста.

Список использованных источников

1. Дыдыкин А. В. Банковские риски как элемент управления финансовой стабильностью банковской деятельности // Финансы и кредит. — 2011. — № 2. — С. 67–70.
2. Frank RG. — [Электронный ресурс]. Режим доступа: <https://frankrg.com/>. — (дата обращения: 10.03.2026).
3. Basel Committee on Banking Supervision. Principles for the Management of Credit Risk // Bank for International Settlements. — 2010. — С. 1–26.
4. Коновалова Н. Assessing Creditworthiness in Consumer Credit // Polish Journal of Management Studies. — 2017. — С. 90–100.
5. Заернюк В. М. Использование скоринговых моделей при планировании периодичности аудита структурных подразделений коммерческого банка // Финансы и кредит. — 2013. — № 7. — С. 28–37.
6. World Bank Group. Credit Scoring Approaches Guidelines // World Bank Group Publications. — 2019. — С. 1–64.
7. Ткачев А. И. Системы кредитного скоринга. Матричный подход // Банковский вестник. — 2019. — С. 37–38.
8. Ляндрес В. А. Технология оценки долгов по потребительским кредитам / В. А. Ляндрес. — Наука Кубани, 2019.
9. Thomas L. C. Credit Scoring and Its Applications / L. C. Thomas, D. B. Edelman, J. N. Crook. — Philadelphia: Society for Industrial and Applied Mathematics, 2002. — С. 172–175. — DOI: 10.1137/1.9780898718317.
10. Корниенко В. Ю. Использование градиентного бустинга деревьев решений для анализа генетических последовательностей / В. Ю. Корниенко. — Аллея науки, 2018. — С. 2–5.
11. Hastie T. The Elements of Statistical Learning: Data Mining, Inference, and Prediction / T. Hastie, R. Tibshirani, J. Friedman. — 2 edition. — New York: Springer, 2009. — С. 337–384. — DOI: 10.1007/978-0-387-84858-7.

12. Салахутдинова К. И. Алгоритм градиентного бустинга деревьев решений в задаче идентификации программного обеспечения // Научно-технический вестник информационных технологий, механики и оптики. — 2018. — Т. 18, № 6. — С. 1016–1022. — DOI: 10.17586/2226-1494-2018-18-6-1016-1022.
13. Chen T. XGBoost: A Scalable Tree Boosting System / T. Chen, C. Guestrin // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. — New York: ACM, 2016. — С. 785–794. — DOI: 10.1145/2939672.2939785.
14. LightGBM: A Highly Efficient Gradient Boosting Decision Tree / G. Ke, Q. Meng, T. Finley et al. // Advances in Neural Information Processing Systems. — 2017. — С. 3146–3154.
15. Пампел Ф. Логистическая регрессия / Ф. Пампел, А. Груздев, Д. Цвиркун. — Москва: ДМК Пресс, 2023. — С. 8–20.
16. Мармыш К. А. Применение логистической регрессии к вычислению повреждаемости твердого деформируемого тела // Механика машин, механизмов и материалов. — 2021. — № 1. — С. 46–53. — DOI: 10.46864/1995-0470-2020-1-54-46-53.
17. Многомерный статистический анализ в экономических задачах: компьютерное моделирование в SPSS / Н. А. Концевая, И. В. Орлова, В. Б. Турундаевский и др.; Под ред. И. В. Орлова. — Москва: Вузовский учебник : ИНФРА-М, 2024. — С. 194–198.
18. Петухов Д. Е. Линейный дискриминантный анализ как контролируемый подход в задачах уменьшения размерности данных // Научное обозрение. Технические науки. — 2020. — С. 5–9.
19. Михайлов И. С. Разработка модификации метода опорных векторов для решения задачи классификации с ограничениями на предметную область // Программные продукты и системы. — 2020. — Т. 33, № 3. — С. 439–448. — DOI: 10.15827/0236-235X.131.439-448.

20. Xiangdong H. Prediction of Bottom-Hole Flow Pressure in Coalbed Gas Wells Based on GA Optimization SVM / H. Xiangdong // 2018 IEEE 3rd Advanced Information Technology, Electronic and Automation Control Conference. — 2018. — С. 138–141. — DOI: 10.1109/IAEAC.2018.8577488.
21. Fair Isaac Corporation. What is a Credit Score. — [Электронный ресурс]. Режим доступа: <https://www.myfico.com/credit-education/credit-scores>. — (дата обращения: 10.03.2026).
22. Чуб В. С. Сравнительный анализ методов машинного обучения в оценке кредитных рисков // Образовательные ресурсы и технологии. — 2023. — № 3. — С. 81–92. — DOI: 10.21777/2500-2112-2023-3-81-92.
23. Molnar C. Interpretable Machine Learning: A Guide for Making Black Box Models Explainable / C. Molnar. — Kiron Open Publishing, 2022. — С. 165–183.
24. Kuhn M. Applied Predictive Modeling / M. Kuhn, K. Johnson. — New York: Springer, 2013. — С. 27–59. — DOI: 10.1007/978-1-4614-6849-3.
25. Bishop C. M. Pattern Recognition and Machine Learning / C. M. Bishop. — New York: Springer, 2006. — С. 32–33.
26. Home Credit Group. Home Credit Default Risk. — [Электронный ресурс]. Режим доступа: <https://www.kaggle.com/c/home-credit-default-risk>. — (дата обращения: 10.03.2026).
27. Davis J. The Relationship Between Precision-Recall and ROC Curves / J. Davis, M. Goadrich // Proceedings of the 23rd International Conference on Machine Learning. — New York: ACM, 2006. — С. 233–240. — DOI: 10.1145/1143844.1143874.

ПРИЛОЖЕНИЕ А

Модуль подготовки данных

Листинг 1 – Функция загрузки исходных данных

```
def load_source_data():
    app = pd.read_csv('application_train.csv')
    bureau = pd.read_csv('bureau.csv')
    bureau_balance = pd.read_csv('bureau_balance.csv')
    previous = pd.read_csv('previous_application.csv')
    installments = pd.read_csv('installments_payments.csv')
    return app, bureau, bureau_balance, previous, installments
```

Листинг 2 – Функция подготовки признаков кредитного бюро

```
def prepare_bureau_features(bureau, bureau_balance):
    bureau_balance['HAS_OVERDUE'] = (
        bureau_balance['STATUS'].isin(['1', '2', '3', '4', '5'])
    ).astype(int)

    balance_agg = bureau_balance.groupby('SK_ID_BUREAU').agg(
        MONTHS_BALANCE_COUNT=('MONTHS_BALANCE', 'count'),
        HAS_OVERDUE_SUM=('HAS_OVERDUE', 'sum')
    )
    bureau = bureau.merge(balance_agg, on='SK_ID_BUREAU', how='left')
    bureau['IS_ACTIVE'] = (bureau['CREDIT_ACTIVE'] == 'Active').astype(int)

    bureau_agg = bureau.groupby('SK_ID_CURR').agg(
        BUREAU_CREDIT_COUNT=('SK_ID_BUREAU', 'count'),
        BUREAU_ACTIVE_COUNT=('IS_ACTIVE', 'sum'),
        BUREAU_DEBT_SUM=('AMT_CREDIT_SUM_DEBT', 'sum'),
        BUREAU_CREDIT_SUM_SUM=('AMT_CREDIT_SUM', 'sum'),
        BUREAU_OVERDUE_MAX=('CREDIT_DAY_OVERDUE', 'max')
    )
    bureau_agg['BUREAU_ACTIVE_RATIO'] = (
        bureau_agg['BUREAU_ACTIVE_COUNT'] /
        bureau_agg['BUREAU_CREDIT_COUNT']
    )
    bureau_agg['BUREAU_HAS_OVERDUE'] = (
        bureau_agg['BUREAU_OVERDUE_MAX'] > 0
    ).astype(int)
    return bureau_agg
```

Листинг 3 – Функция подготовки признаков предыдущих заявок

```
def prepare_previous_features(previous):
    previous['IS_APPROVED'] = (
        previous['NAME_CONTRACT_STATUS'] == 'Approved'
    ).astype(int)
    previous['IS_REFUSED'] = (
        previous['NAME_CONTRACT_STATUS'] == 'Refused'
    ).astype(int)
    previous_agg = previous.groupby('SK_ID_CURR').agg(
        PREV_APP_COUNT=('SK_ID_PREV', 'count'),
        PREV_APPROVED_COUNT=('IS_APPROVED', 'sum'),
        PREV_REFUSED_COUNT=('IS_REFUSED', 'sum'),
        PREV_CREDIT_MEAN=('AMT_CREDIT', 'mean'),
        PREV_ANNUITY_MEAN=('AMT_ANNUITY', 'mean')
    )
    previous_agg['APPROVAL_RATIO'] = (
        previous_agg['PREV_APPROVED_COUNT'] /
        previous_agg['PREV_APP_COUNT']
    )

    return previous_agg
```

Листинг 4 – Функция подготовки признаков платежного поведения

```
def prepare_installments_features(installments):
    installments['LATE'] = (
        installments['DAYS_ENTRY_PAYMENT'] >
        installments['DAYS_INSTALMENT']
    ).astype(int)

    installments['DELAY'] = (
        installments['DAYS_ENTRY_PAYMENT'] -
        installments['DAYS_INSTALMENT']
    )
    installments_agg = installments.groupby('SK_ID_CURR').agg(
        INSTALMENTS_COUNT=('SK_ID_PREV', 'count'),
        INSTALMENTS_LATE_COUNT=('LATE', 'sum'),
        INSTALMENTS_DELAY_MEAN=('DELAY', 'mean'),
        INSTALMENTS_DELAY_MAX=('DELAY', 'max'),
        INSTALMENTS_PAYMENT_MEAN=('AMT_PAYMENT', 'mean'),
        INSTALMENTS_INSTALMENT_MEAN=('AMT_INSTALMENT', 'mean')
    )
    installments_agg['INSTALMENTS_LATE_RATIO'] = (
        installments_agg['INSTALMENTS_LATE_COUNT'] /
        installments_agg['INSTALMENTS_COUNT']
    )

    return installments_agg
```

Листинг 5 – Функция формирования производных признаков

```
df['AGE_YEARS'] = -df['DAYS_BIRTH'] / 365
df['DAYS_EMPLOYED'] = df['DAYS_EMPLOYED'].replace(365243, 0)
df['EMPLOYED_YEARS'] = -df['DAYS_EMPLOYED'] / 365
df['EMPLOYED_TO_AGE'] = df['EMPLOYED_YEARS'] / (df['AGE_YEARS'] + 1)

df['ACTIVE_CREDIT_RATIO'] = (
    df['BUREAU_ACTIVE_COUNT'] / (df['BUREAU_CREDIT_COUNT'] + 1)
)
df['DEBT_TO_CREDIT'] = (
    df['BUREAU_DEBT_SUM'] / (df['BUREAU_CREDIT_SUM_SUM'] + 1)
)
df['REFUSAL_RATIO'] = df['PREV_REFUSED_COUNT'] / (df['PREV_APP_COUNT'] + 1)
df['HAS_REFUSALS'] = (df['PREV_REFUSED_COUNT'] > 0).astype(int)

return df
```

ПРИЛОЖЕНИЕ Б

Модуль обучения, тестирования и предсказания модели

Листинг 6 – Параметры обучения модели

```
{
    "test_size": 0.2,
    "random_state": 42,
    "threshold": 0.3,
    "cv": 3,
    "scoring": "roc_auc",
    "random_search_iter": 5,
    "n_estimators": [300, 500, 700],
    "learning_rate": [0.01, 0.03, 0.05],
    "num_leaves": [31, 50, 70],
    "max_depth": [6, 8],
    "min_child_samples": [50, 100]
}
```

Листинг 7 – Функция подготовки обучающей и тестовой выборки

```
def prepare_train_test_data(dataset_path, settings):
    df = pd.read_csv(dataset_path)

    y = df['TARGET']
    X = pd.get_dummies(df.drop(columns=['TARGET', 'SK_ID_CURR']))

    X.columns = X.columns.str.replace('_', '_')

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=settings['test_size'],
        random_state=settings['random_state'],
        stratify=y
    )

    return X_train, X_test, y_train, y_test, X.columns
```


Листинг 8 – Функция обучения модели LightGBM

```
def train_lightgbm_model(X_train, y_train, feature_columns, settings):
    scale_pos_weight = len(y_train[y_train == 0]) / len(y_train[y_train == 1])

    base_model = LGBMClassifier(
        scale_pos_weight=scale_pos_weight,
        random_state=settings['random_state'],
        n_jobs=-1,
        force_col_wise=True
    )

    param_grid = {
        name: settings[name]
        for name in [
            'n_estimators',
            'learning_rate',
            'num_leaves',
            'max_depth',
            'min_child_samples'
        ]
    }

    search = RandomizedSearchCV(
        base_model, param_grid,
        n_iter=settings['random_search_iter'],
        scoring=settings['scoring'],
        cv=settings['cv'],
        random_state=settings['random_state'],
        n_jobs=-1
    )

    search.fit(X_train, y_train)
    model = search.best_estimator_

    joblib.dump(model, 'model.pkl')
    joblib.dump(feature_columns, 'columns.pkl')

    return model
```

Листинг 9 – Функция оценки качества модели

```
from sklearn.metrics import (
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score
)

def evaluate_model(model, X_test, y_test, threshold=0.3):
    y_proba = model.predict_proba(X_test)[: , 1]
    y_pred = (y_proba > threshold).astype(int)
    metrics = {
        'ROC-AUC': roc_auc_score(y_test, y_proba),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred),
        'F1-score': f1_score(y_test, y_pred)
    }
    return metrics
```

Листинг 10 – Функция формирования результата скоринга

```
def predict_client_risk(model, input_df, columns):
    input_df = pd.get_dummies(input_df)
    input_df = input_df.reindex(columns=columns, fill_value=0)
    probability_default = model.predict_proba(input_df)[0][1]
    score = (1 - probability_default) * 1000

    if probability_default < 0.3:
        risk_level = 'Low'
    elif probability_default < 0.6:
        risk_level = 'Medium'
    else:
        risk_level = 'High'
    result = {
        'probability_default': float(probability_default),
        'score': float(score),
        'risk_level': risk_level,
        'decision': int(probability_default >= 0.5)
    }
    return result
```

ПРИЛОЖЕНИЕ В

Презентация состоит из 18 слайдов.